CHAPTER 1

INTRODUCTION

1.1 Background and motivation

The rapid growth of digital payment systems has significantly increased the risk of credit card fraud, leading to billions of dollars in global losses each year and making fraud detection a critical application of machine learning. A key challenge in this domain is extreme class imbalance, where fraudulent transactions represent only a tiny fraction of the data, along with highly asymmetric misclassification costs, as failing to detect fraud can result in substantial financial damage.

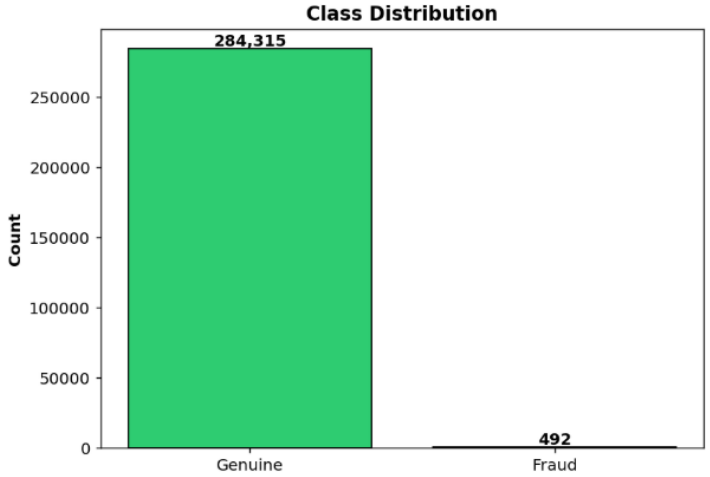

Fig 1.1: Extreme Class Imbalance in Financial Transactions

Classical machine learning models such as Support Vector Machines, Random Forests, and gradient boosting techniques have been widely used for fraud detection and have demonstrated strong performance. However, these models operate within fixed feature spaces and may struggle to capture complex, high-order relationships in large-scale transaction data.

Quantum machine learning (QML) offers a promising alternative by encoding classical data into quantum states, enabling computation in exponentially large feature spaces. This theoretical advantage motivates the exploration of quantum models for fraud detection tasks.

1.2 Objectives of the Project

The main objectives of this project are:

- To implement and evaluate classical and quantum machine learning models for credit card fraud detection

- To ensure a fair comparison by training all models on a shared dataset under identical conditions
- To study the impact of noise on quantum model performance under simulated conditions
- To assess the practical applicability of quantum machine learning in fraud detection

1.3 Scope of the Project

- The scope of this project is limited to binary classification of credit card transactions as fraudulent or genuine using the Kaggle Credit Card Fraud Detection dataset.
- The study focuses on a data-constrained setting, using a balanced dataset of 200 samples with four PCA-reduced features. Quantum models are implemented using Qiskit and executed on a classical simulator due to hardware limitations.
- The project does not involve real-time deployment or integration with financial systems but focuses on experimental evaluation and comparative analysis.

1.4 Significance of the Project

This project contributes to understanding the practical capabilities and limitations of quantum machine learning in real-world applications. By ensuring a fair comparison framework, it provides meaningful insights into whether quantum models offer any advantage over classical approaches. The introduction of QABD analysis helps identify specific conditions under which quantum models may perform competitively. Additionally, noise analysis highlights the challenges associated with current quantum hardware, making the study relevant for future advancements in the field.

1.5 Reason for choosing the Project

The motivation behind this project is to explore emerging technologies such as quantum machine learning in the context of real-world problems like fraud detection.
With the increasing importance of secure financial systems, it is essential to evaluate new approaches that may enhance detection capabilities. This project aims to bridge the gap between theoretical advantages of quantum computing and its practical applicability, providing a deeper understanding of its current potential.

CHAPTER 2

LITERATURE REVIEW

2.1 Existing Work

This chapter presents a review of recent studies related to quantum machine learning and its applications in classification and fraud detection. It highlights significant contributions in quantum feature mapping, kernel-based methods, variational quantum models, and hybrid approaches for handling complex and imbalanced datasets. The review focuses on understanding how quantum-enhanced techniques improve data representation, learning efficiency, and model performance, while also identifying key challenges such as scalability and dependency on dataset characteristics.

M. Schuld and N. Killoran [1], introduced the concept of quantum machine learning in feature Hilbert spaces, providing a strong theoretical framework for encoding classical data into quantum states. Their work explained how quantum feature maps project data into exponentially large spaces, enabling better separability. This approach forms the foundation for quantum kernel methods like QSVM. It highlights the potential advantage of quantum systems in handling complex, high-dimensional data.

V. Havlíček et al. [2], proposed a supervised learning model using quantum-enhanced feature spaces, where parameterized quantum circuits encode classical data. Their study demonstrated that quantum feature maps can capture highly non-linear relationships. They showed that quantum models can outperform classical counterparts in specific scenarios. This work is considered a key milestone in practical quantum machine learning.

D. Alvarez-Estevez [3], conducted benchmarking experiments on quantum kernel-based models across multiple classification tasks. The study compared different kernel training techniques and their impact on performance. Results indicated that model success depends heavily on dataset structure and kernel selection. It emphasized the importance of proper tuning in quantum models.

J. R. Glick et al. [4], introduced covariant quantum kernels tailored for datasets with inherent group structures. Their work incorporated symmetry properties into quantum feature maps to improve learning efficiency. This approach enhanced generalization and reduced redundancy in learning. It is particularly useful for structured and relational datasets.

N. Innan et al. [5], presented a comparative study of various quantum machine learning models

applied to financial fraud detection. Models such as QSVM and VQC were evaluated against classical approaches. The study showed that quantum models can perform competitively even with fewer features. It highlighted the feasibility of quantum methods in financial applications.

M. El Alami et al. [6], analyzed the performance of different quantum machine learning architectures for credit card fraud detection. Their work compared hybrid and fully quantum models on real datasets. Results showed that hybrid approaches often provide better stability and accuracy. The study also discussed challenges like noise and scalability.

F. Atban et al. [7], proposed improving variational quantum classifier performance using meta-heuristic feature selection techniques. Their approach reduced irrelevant features and enhanced model efficiency. Experimental results showed improved accuracy and reduced training complexity. This work highlights the importance of feature optimization in QML.

C. Huot et al. [8], developed a quantum autoencoder model for fraud detection in highly imbalanced datasets. Their method focused on compressing and reconstructing data to detect anomalies. The approach improved feature representation and detection capability. It demonstrated strong performance in identifying rare fraud cases.

N. Innan et al. [9], explored fraud detection using quantum graph neural networks, integrating relational data into quantum models. Their work showed that graph-based structures can capture complex transaction relationships. This approach improved detection of fraud patterns in interconnected systems. It represents an advanced direction in quantum deep learning.

J. M. Neto and G. P. Temporão [10], proposed a quantum-assisted restricted Boltzmann machine for fraud detection. Their hybrid model combined quantum sampling with classical learning techniques. The study showed improved pattern recognition capabilities. It demonstrated the potential of quantum-enhanced probabilistic models.

T. Hubregtsen et al. [11], investigated training quantum embedding kernels on near-term quantum devices. Their work addressed practical challenges such as noise, limited qubits, and hardware constraints. They proposed optimization strategies to improve performance. This study is important for real-world implementation of QML models.

A. Di Pierro and M. Incudini [12], implemented quantum machine learning techniques for credit card fraud detection. Their study demonstrated how quantum algorithms can be applied in real financial scenarios. Results showed promising performance compared to classical methods. It highlighted the practical feasibility of QML integration.

F. K. Alarfaj et al. [13], analyzed state-of-the-art machine learning and deep learning models for fraud detection. Their study provided a strong baseline comparison for evaluating quantum models. They discussed model performance, scalability, and feature handling. This work is useful for understanding classical benchmarks.

I. D. Mienye and N. Jere [14], presented a detailed review of deep learning approaches for credit card fraud detection. The study covered challenges such as data imbalance and feature engineering. It also discussed model interpretability and deployment issues. This work provides a comprehensive understanding of existing techniques.

H. Fanai and H. Abbasimehr [15], proposed a hybrid model combining deep autoencoders with classification networks. Their approach improved feature extraction and anomaly detection. Experimental results showed enhanced fraud detection accuracy. It demonstrated the effectiveness of combining representation learning with classification.

G. Zioviris et al. [16], developed a multistage deep learning model for fraud detection. Their framework applied multiple models sequentially to improve detection performance. This approach increased accuracy and reduced false positives. It highlights the benefits of layered model architectures.

S. K. Hashemi et al. [17], explored ensemble learning techniques combined with SMOTE oversampling for fraud detection. Their work addressed the issue of class imbalance effectively. Results showed improved recall and overall performance. It demonstrated the importance of data balancing techniques.

P. Kumar et al. [18], conducted a systematic review of machine learning approaches for credit card fraud detection. The study summarized various algorithms, datasets, and evaluation methods. It identified key challenges and future research directions. This work provides a broad overview of the field.

2.1.1 Analysis of Existing Methods

From the reviewed studies, it is evident that quantum machine learning is emerging as a promising approach for solving classification problems, particularly in fraud detection. Quantum kernel-based methods such as QSVM provide stable performance by leveraging high-dimensional feature spaces, while variational models such as VQC introduce flexibility through trainable parameters but often face optimization challenges.

Classical machine learning models continue to serve as strong baselines due to their robustness and ability to handle tabular datasets effectively. Several studies also highlight the importance

of preprocessing techniques such as dimensionality reduction and feature selection in improving model performance. However, most existing works evaluate quantum and classical models under different experimental settings, leading to inconsistent and often biased comparisons. Additionally, there is limited exploration of how dataset size, feature dimensionality, and noise affect the performance of quantum models.

2.1.2 Need for the Proposed System

To address these limitations, there is a need for a structured and fair evaluation framework that compares classical and quantum models under identical conditions. This project proposes a methodology where all models are trained on the same dataset with equal sample size and feature dimensions, ensuring unbiased comparison. The introduction of Quantum Advantage Boundary Detection (QABD) enables systematic analysis of model performance across different configurations.

Additionally, the use of trainable quantum kernel methods such as Quantum Kernel Alignment (QKA) improves adaptability and enhances classification performance. The inclusion of noise robustness analysis further provides insights into the feasibility of deploying quantum models in real-world scenarios. Overall, this work aims to provide a comprehensive and realistic evaluation of quantum machine learning for credit card fraud detection.

2.2 Research Gaps

1. **Unfair comparisons between classical and quantum models.**

   Our Approach: We addressed this by implementing a "Fair Same-Data Comparison" framework, where both classical (e.g., SVM-RBF, Random Forest) and quantum models (QSVM, QKA) were trained and evaluated on identical 200-sample subsets extracted from the same credit card fraud dataset.

2. **Reliance on a single train-test split without cross-validation.**

   Our Approach: To ensure the reliability of our results, we implemented 5-Fold Cross-Validation (CV) across all models. This methodology eliminated evaluation bias and confirmed that the performance of the Quantum Kernel Alignment (QKA) model was consistent across different data folds.

3. **Lack of systematic analysis regarding dataset size and feature dimensionality.**

   Our Approach: We conducted a systematic Quantum Advantage Boundary Detection (QABD) analysis. This allowed us to map quantum performance as a function of sample size

and feature count, identifying specific regimes where quantum methods begin to outperform classical baselines.

4. **Fixed quantum feature maps versus trainable kernel methods.**

   Our Approach: Instead of relying solely on static feature maps, our project explored trainable quantum kernel methods. Specifically, we utilized Quantum Kernel Alignment (QKA) to optimize the quantum kernel for our specific fraud dataset, which resulted in superior F1-scores and perfect precision compared to non-trainable counterparts.

5. **Neglect of the impact of noise on NISQ-era quantum systems.**

   Our Approach: We performed a detailed noise robustness analysis by introducing depolarizing error probabilities into our simulations. Our findings identified a NISQ deployment threshold at approximately 5% noise, providing critical insights into the real-world applicability of these models.

2.3 Problem Statement

Credit card fraud detection is inherently challenging due to extreme class imbalance and the need for high-dimensional pattern recognition. In real-world datasets, fraudulent transactions account for less than 0.2% of all records, making standard accuracy ineffective and shifting the focus to precision, recall, and F1-score as reliable evaluation metrics. Classical machine learning models such as SVM, Random Forest, and XGBoost have demonstrated strong performance in this domain.

Quantum machine learning offers a theoretically powerful alternative. By encoding data into quantum states using parameterized circuits, quantum models operate in exponentially scaling Hilbert spaces, enabling them to capture intricate non-linear relationships that may be inaccessible to classical approaches. Despite this potential, existing research often lacks fair comparison, as classical and quantum models are typically trained on different dataset sizes. This leads to biased results, where performance differences reflect data advantages rather than true algorithmic capability.

This project addresses that gap using the Kaggle Credit Card Fraud Detection dataset (284,807 transactions, including only 492 fraudulent cases), ensuring both classical and quantum models are trained on identical data. The goal is to evaluate whether quantum models can match or outperform classical methods under equal conditions and to identify the dataset sizes and feature dimensions where quantum advantage may emerge.

CHAPTER 3

METHODOLOGY / SYSTEM DESIGN

3.1 Existing System

Current approaches to quantum-classical comparison in fraud detection suffer from a fundamental methodological flaw — classical and quantum models are evaluated on different training data. Classical models, unconstrained by qubit limitations, are trained on thousands of samples, while quantum models are restricted to a few hundred due to circuit simulation overhead. Any observed performance gap under such conditions reflects a data advantage rather than a genuine algorithmic difference, making conclusions about quantum competitiveness unreliable.

Some common limitations of existing systems include:

- Unfair evaluation where classical models use larger datasets than quantum models
- Lack of cross-validation for quantum models, leading to unstable results
- No systematic analysis of conditions for quantum advantage
- Limited exploration of trainable quantum kernel methods (e.g., QKA)
- No analysis of noise robustness for NISQ deployment

3.2 Proposed System

This project proposes a fair and reproducible quantum-classical comparison pipeline that directly addresses the shortcomings of existing studies. The solution enforces identical training conditions for all nine models and introduces three novel analytical contributions not present in prior literature.

The system is composed of the following elements:

- A structured preprocessing workflow using StandardScaler, SMOTE, and PCA to create a consistent dataset for both classical and quantum models
- Implementation of four classical models (SVM, XGBoost, Random Forest, Logistic Regression) and five quantum models (QSVM-ZZ, QSVM-Pauli, VQC, QNN, QKA)
- A fair evaluation setup using five-fold stratified cross-validation applied uniformly across all models

- An experimental analysis (QABD) by varying sample size and feature dimensions to study model performance
- Simulation of depolarizing noise to observe its effect on quantum model performance
- Performance evaluation using Precision, Recall, F1-score, ROC-AUC, and PR-AUC

3.3 Requirements

3.3.1 Software Requirements

- **Programming Languages:** Python 3x
- **Libraries/Frameworks:**
  - Qiskit 1.2.4 (quantum circuit construction and execution)
  - Qiskit-Aer 0.15.1 (statevector simulator backend)
  - Qiskit Machine Learning 0.7.2 (QSVM, VQC, QNN)
  - Qiskit Algorithms 0.3.1 (COBYLA optimiser, quantum kernel alignment)
  - Scikit-learn (preprocessing, classical models, cross-validation)
  - Imbalanced-learn (SMOTE for class balancing)
  - XGBoost (gradient-boosted classification)
  - SHAP (feature importance analysis)
  - Pandas and NumPy (data handling and numerical operations)
  - Matplotlib and Seaborn (visualisation)
- **Jupyter Notebook / VS Code** (for development and debugging)
- **Operating System:** Windows/Linux

3.3.2 Hardware Requirements

- **Processor:** Minimum Intel i5 or equivalent
- **RAM:** Minimum 8 GB (16 GB recommended)
- **Storage:** Minimum 500 MB of free disk space
- **GPU (Optional):** Not required — quantum simulation is CPU-based via statevector backend.

3.4 System Architecture

The architecture is structured as a sequential workflow that branches into two parallel tracks one classical and one quantum before converging at a unified evaluation stage.

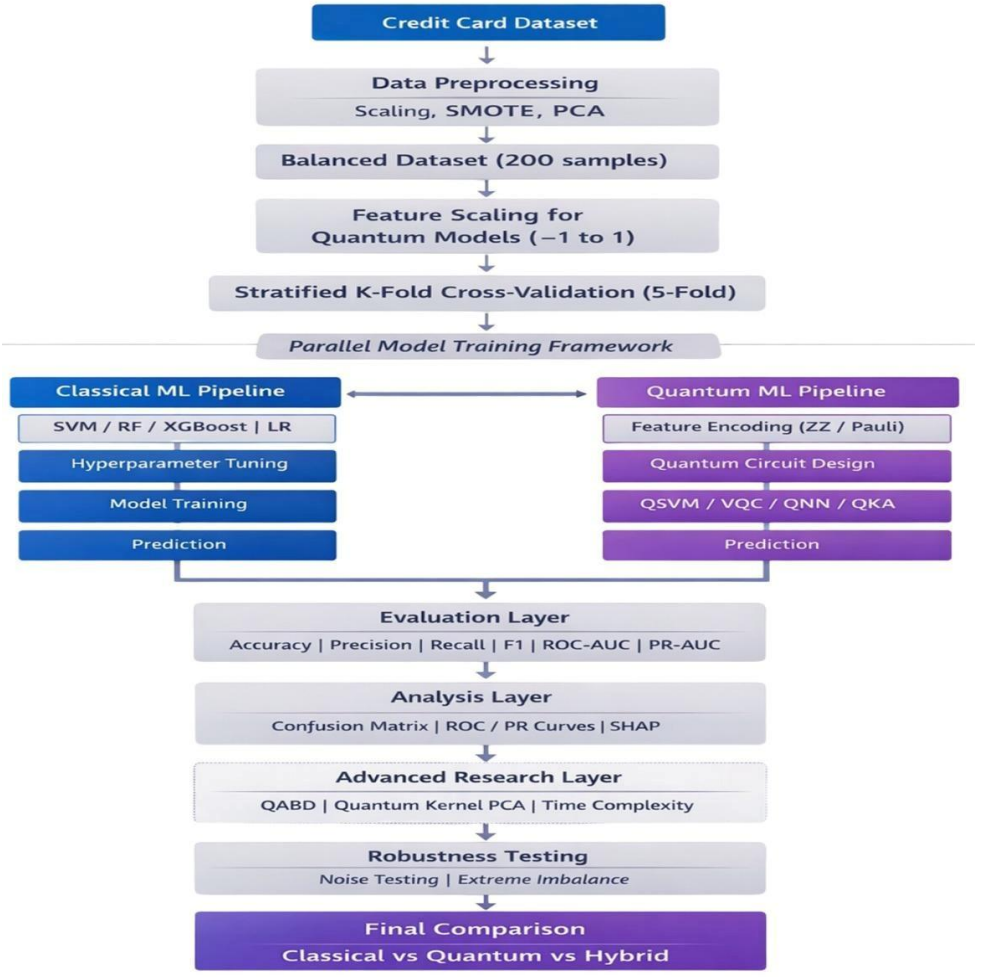

Fig 3.4: Architecture for Quantum and Classical Model Comparison

The architecture is structured as a sequential workflow that branches into two parallel tracks one classical and one quantum — before converging at a unified evaluation stage. Figure 3.4 illustrates this end-to-end flow.

The raw Kaggle Credit Card Fraud Detection dataset, containing 284,807 transactions, is first loaded into the system. It then passes through a preprocessing stage involving scaling, class balancing using SMOTE, and dimensionality reduction using PCA, which reduces 30 features to four principal components while retaining approximately 86% of the original variance.

This process produces a shared training dataset consisting of 200 balanced samples with four features. This dataset is provided identically to both the classical and quantum model branches. Classical models include SVM (RBF), XGBoost, Random Forest, and Logistic Regression, while quantum models include QSVM (ZZFeatureMap), QSVM (PauliFeatureMap), VQC, QNN, and QKA.

To ensure fair evaluation, all models are trained and tested under consistent conditions using the same dataset and identical data splits. Five-fold stratified cross-validation is applied uniformly across all models to minimize evaluation bias and improve reliability of results. The outputs from all models are further used for additional analyses, including performance variation under different data configurations and noise robustness evaluation, before drawing final conclusions.

3.5 Proposed Model

3.5.1 Dataset

The dataset used in this project is the Kaggle Credit Card Fraud Detection dataset, a widely used benchmark for imbalanced binary classification. It contains transactions made by European cardholders over two days in September 2013.

Official Link:https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud

The dataset consists of 284,807 transactions, of which only 492 are fraudulent (0.1727%), making it highly imbalanced. This imbalance is a key challenge and is addressed using SMOTE during preprocessing.

3.5.2 Features

The dataset consists of 30 features, including 28 anonymized PCA-generated variables (V1–V28), Time, and Amount. The target variable, Class, identifies transactions as genuine (0) or fraudulent (1).To accommodate the qubit limitations of quantum hardware, a two-stage

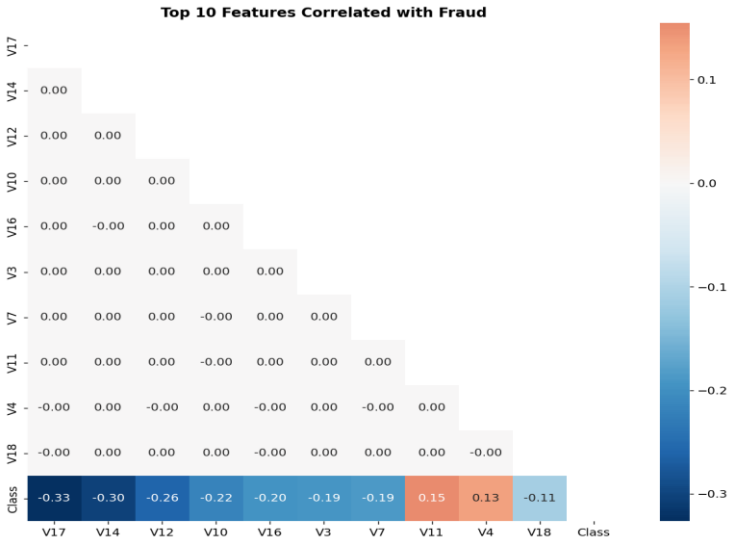

Fig 3.5.2: Feature Correlation Ranking

dimensionality reduction is performed. First, the top 10 features most strongly correlated with fraud are identified (Fig 3.2). These are then compressed via a secondary PCA into four principal components, retaining 86% of the data variance while ensuring minimal quantum circuit depth.

3.5.3 Data Preprocessing

A structured preprocessing workflow is applied to ensure consistency, eliminate bias, and prepare the classical data for quantum encoding. The pipeline consists of the following sequential steps:

1. **Stratified Train-Test Split**
   The dataset is first divided into training and testing sets using an 80/20 stratified split. Stratification guarantees that the original, highly imbalanced class distribution is proportionally preserved in both subsets, ensuring the model is evaluated on a realistic distribution of fraudulent and genuine transactions.
2. **Feature Standardization (StandardScaler)**
   To prevent features with larger numeric ranges (such as Amount) from dominating the objective function, all feature values are standardized to possess a mean of zero and a variance of one. This is critical for distance-based classical algorithms (like SVM) and stable PCA projection.

$$z = \frac{(x - \mu)}{\sigma}$$

   where μ is the sample mean and σ is the standard deviation of the training set.
3. **Class Balancing via SMOTE**
   To address the extreme class imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) is applied exclusively to the training data to prevent data leakage. SMOTE generates synthetic fraudulent transactions by interpolating between existing minority instances and their k-nearest neighbors, creating a perfectly balanced training environment.

$$x_{\{new\}} = x_i + \lambda \left(x_{\{zi\}} - x_i\right)$$

   where x_i is a minority class sample, x_{zi} is a randomly chosen nearest neighbor of x_i, and λ is a random interpolation weight drawn from Uniform(0, 1).

4. **Dimensionality Reduction (PCA)**

   Current NISQ-era (Noisy Intermediate-Scale Quantum) hardware is heavily constrained by qubit counts and circuit depth. To accommodate this, Principal Component Analysis (PCA) is applied to reduce the high-dimensional feature space down to exactly four principal components. This step retains approximately 86% of the data variance while allowing the data to be encoded onto a 4-qubit quantum register.

$$\boldsymbol{C\,v = \lambda\,v}$$

   where C is the covariance matrix, v represents eigenvectors, and λ represents eigenvalues. The data is projected onto the eigenvectors corresponding to the largest eigenvalues.

5. **Quantum Angle Scaling (MinMaxScaler)**

   Because the quantum models (QSVM, VQC, QNN) encode classical data into quantum states using parameterized rotational gates (such as R_y and R_z), the input data must represent valid rotation angles. A MinMaxScaler is subsequently applied to bound the four PCA features within the range [-1, 1].

$$\boldsymbol{x_{\{scaled\}} = 2\left(x - x_{\{min\}}\right)/x_\{max\} - x_\{min\}) - 1}$$

   Following this rigorous preprocessing pipeline, a shared subset of 200 samples (100 genuine and 100 fraudulent) is extracted. This dataset serves as the strict, common training input for both the classical and quantum models to ensure a perfectly fair same-data comparison.

CHAPTER 4

IMPLEMENTATION & TESTING

This chapter presents the complete implementation of the proposed comparative framework integrating classical and quantum machine learning models for credit card fraud detection. The implementation is designed to ensure experimental fairness, reproducibility, and consistency across all models by maintaining a unified preprocessing and evaluation pipeline.

Technologies Used

The following tools and technologies were used for the development and implementation of the system.

Table 4: Technologies Used in Implementation

Technology	Purpose
Python 3.x	The primary programming language used for the implementation
Pandas	Facilitates efficient data manipulation and dataset handling
NumPy	Provides support for numerical operations and matrix computations
Scikit-learn	Used for preprocessing, PCA, classical models, and cross-validation
XGBoost	Implements gradient boosting for classification tasks
Imbalanced-learn	Used for SMOTE-based class balancing
Qiskit (v1.2.4)	Used for quantum circuit design and model implementation
Qiskit Aer	Provides simulation backend for executing quantum circuits
Qiskit Machine Learning (v0.7.2)	Implements QSVM, VQC, QNN, and quantum models
SciPy	Used for optimization (COBYLA) in variational models
SHAP	Used for feature importance analysis
Matplotlib & Seaborn	Libraries for visualization and graphical analysis
Jupyter Notebook	Environment used for development, experimentation, and testing

4.1 Implementation Approach

The implementation follows a structured pipeline consisting of data acquisition, preprocessing, model training, and evaluation. The Kaggle Credit Card Fraud Detection dataset is used, containing 284,807 transactions with extreme class imbalance, where fraudulent transactions constitute approximately 0.172% of the data. This imbalance makes fraud detection a challenging classification problem.

To address this issue, a shared preprocessing pipeline is applied to all models to ensure consistency. The preprocessing steps are described as follows:

- **Feature Scaling:** The dataset is normalized using scaling techniques to ensure uniform feature distribution across all attributes.
- **Class Balancing**: SMOTE (Synthetic Minority Over-sampling Technique) is applied to generate synthetic fraud samples and balance the dataset.
- **Dimensionality Reduction:** Principal Component Analysis (PCA) is used to reduce the feature space from 30 features to 4 principal components, while retaining approximately 86% of the total variance.

A balanced dataset of 200 samples (100 fraudulent and 100 genuine transactions) is constructed and used uniformly across all models. This ensures a fair comparison by eliminating bias caused by varying dataset sizes.

All models are evaluated using five-fold stratified cross-validation, ensuring that each fold maintains class balance and that performance estimates are reliable and robust.

4.2 Functional and Algorithmic Implementation

The proposed framework integrates a total of nine models, including four classical machine learning models and five quantum machine learning models. All models are trained and evaluated under a unified experimental setup to maintain fairness in comparison.

Each model is implemented using its respective computational framework. Classical models are developed using Scikit-learn and XGBoost libraries, while quantum models are implemented using Qiskit and executed on Aer simulators. Optimization routines for quantum models are handled using classical optimizers such as COBYLA.

Models Used:

4.2.1 Classical Baseline Models

4.2.1.1 Support Vector Machine (SVM-RBF Kernel)

Model Overview:

The Support Vector Machine (SVM) with Radial Basis Function (RBF) kernel is a classical supervised machine learning algorithm designed for binary classification tasks. It is particularly effective in handling datasets where the relationship between features and class labels is non-linear. In the context of credit card fraud detection, transaction patterns are often complex and cannot be separated using simple linear boundaries.

In this study, SVM serves as a strong baseline model against which the performance of quantum machine learning models is evaluated. Its ability to construct optimal decision boundaries in transformed feature spaces makes it highly suitable for fraud detection tasks involving subtle variations between fraudulent and legitimate transactions.

Mathematical Foundation:

The fundamental objective of SVM is to identify an optimal hyperplane that maximizes the margin between two classes. For linearly separable data, this hyperplane is defined as:

$$\mathbf{w} \cdot \mathbf{x} + \mathbf{b} = \mathbf{0}$$

However, in real-world datasets such as fraud detection, the data is not linearly separable. To address this, SVM employs the kernel trick, which maps the input data into a higher-dimensional feature space where linear separation becomes possible.

This kernel measures similarity between data points based on their distance. The parameter γ controls the spread of the kernel and determines how much influence a single training example has.

The optimization problem solved by SVM is: $\min (1/2 \, ||w||^2 + C \, \Sigma \, \xi_i)$

Working Mechanism:

The SVM model processes input data after preprocessing steps such as scaling and PCA transformation. Each transaction is represented as a point in a reduced feature space consisting of four principal components.

The RBF kernel computes similarity between all pairs of data points, effectively transforming the feature space into a higher-dimensional representation. In this transformed space, the algorithm identifies a hyperplane that maximizes the separation between fraudulent and non-fraudulent transactions.

Support vectors are the data points closest to the decision boundary and play a crucial role in defining the hyperplane. Only these points influence the final model, making SVM memory efficient.

The classification decision is made based on the sign of the decision function, which indicates whether a transaction lies on the fraud or non-fraud side of the boundary.

Algorithm:

Step 1: Data Preprocessing

The input dataset undergoes normalization using feature scaling to ensure all features contribute equally. PCA is applied to reduce dimensionality and remove noise.

Step 2: Kernel Transformation

The RBF kernel function is applied to compute similarity between each pair of data points, generating a kernel matrix.

Step 3: Formulation of Optimization Problem

The model formulates a quadratic optimization problem aimed at maximizing the margin between classes while minimizing classification errors.

Step 4: Identification of Support Vectors

Data points that lie closest to the decision boundary are identified as support vectors. These points are critical in defining the hyperplane.

Step 5: Model Training

The optimization problem is solved using quadratic programming techniques, resulting in optimal values for parameters defining the hyperplane.

Step 6: Decision Boundary Construction

Using the support vectors and optimized parameters, the model constructs a decision boundary in the transformed feature space.

Step 7: Prediction Phase

For new input data, the model computes kernel similarity with support vectors and evaluates the decision function to classify the transaction.

Step 8: Probability Estimation

If probability estimation is enabled, the model converts decision scores into probability values using calibration methods.

Training and Data Handling:

The SVM model is trained on a balanced dataset generated using SMOTE, ensuring equal representation of fraudulent and non-fraudulent transactions. This is critical in fraud detection, where class imbalance can bias the model toward majority classes.

Feature scaling ensures that all features are on a comparable scale, preventing dominance of any single feature. PCA reduces dimensionality from 30 features to 4 components, retaining most of the variance while simplifying the model.

This preprocessing pipeline ensures that the SVM model receives clean, normalized, and informative input data, improving both training efficiency and prediction accuracy.

Evaluation Metrics:

Model performance is evaluated using Precision, Recall, and F1 Score to measure prediction accuracy and balance, while ROC-AUC and PR-AUC assess classification effectiveness, especially on imbalanced fraud data.

Results and Analysis:

The SVM model demonstrates strong performance in detecting non-linear patterns within transaction data. It achieves a balanced trade-off between precision and recall, making it effective in identifying fraudulent transactions without generating excessive false positives.

Compared to simpler models such as Logistic Regression, SVM provides improved performance due to its ability to model complex relationships. It serves as a reliable classical baseline for evaluating the performance of quantum models.

4.2.1.2 XGBoost (Extreme Gradient Boosting)

Model Overview:

XGBoost (Extreme Gradient Boosting) is an advanced ensemble machine learning algorithm based on gradient boosting principles, designed to achieve high predictive accuracy and computational efficiency. It is particularly effective for structured data problems such as credit card fraud detection, where complex relationships exist between input features.

In this project, XGBoost is employed as one of the primary classical models due to its ability to model non-linear interactions, handle feature dependencies, and incorporate regularization to prevent overfitting. Unlike simple decision tree models, XGBoost builds multiple trees sequentially, where each tree learns from the errors of the previous ones. This iterative improvement makes it highly suitable for detecting subtle fraud patterns that may not be captured by individual models.

Within the proposed framework, XGBoost serves as a high-performance benchmark among classical models, providing a strong point of comparison for evaluating whether quantum machine learning approaches offer any practical advantage.

Mathematical Foundation:

XGBoost is based on an additive model where the final prediction is obtained by summing the outputs of multiple decision trees:

$$\hat{y} = \Sigma\, f_k(x)$$

where each f_k represents an individual decision tree.

The model minimizes a regularized objective function defined as:

$$\mathbf{Obj} = \Sigma\, L(y_i, \hat{y}_i) + \Sigma\, \Omega(f_k)$$

where:

$L(y_i, \hat{y}_i)$ is the loss function (e.g., logistic loss for classification)

$\Omega(f_k)$ is the regularization term controlling model complexity

The optimization is performed using gradient descent, where first-order and second-order derivatives of the loss function are used to update model parameters efficiently.

Working Mechanism:

XGBoost operates by constructing decision trees sequentially, where each new tree focuses on correcting the residual errors of the previous trees. Initially, the model starts with a simple prediction, and at each iteration, it computes the gradient of the loss function to determine how predictions should be adjusted.

The algorithm uses a greedy approach to split nodes in each tree based on feature values that maximize information gain. It also incorporates regularization techniques such as L1 and L2 penalties to avoid overfitting, which is particularly important in fraud detection tasks.

In this implementation, XGBoost processes PCA-reduced features and learns patterns that distinguish fraudulent transactions from genuine ones by analyzing feature interactions and distribution differences.

Algorithm:

Step 1: Data Preprocessing

The dataset is normalized using scaling techniques and reduced using PCA. SMOTE is applied to balance class distribution.

Step 2: Initialization

Initialize model predictions with a constant value, typically the mean of the target variable.

Step 3: Gradient Computation

Compute the first-order (gradient) and second-order (Hessian) derivatives of the loss function for each data point.

Step 4: Tree Construction

Build a decision tree by selecting splits that maximize gain based on gradient and Hessian values.

Step 5: Leaf Weight Calculation

Assign optimal weights to leaf nodes using second-order optimization.

Step 6: Model Update

Update predictions by adding the output of the newly constructed tree.

Step 7: Regularization

Apply penalties to control tree complexity and prevent overfitting.

Step 8: Iterative Learning

Repeat steps 3–7 for a predefined number of trees (estimators).

Step 9: Prediction Phase

Aggregate outputs from all trees to generate final prediction probabilities.

Training and Data Handling:

The XGBoost model is trained on a balanced dataset generated using SMOTE, ensuring equal representation of fraud and non-fraud cases. Feature scaling ensures numerical consistency, and PCA reduces dimensionality while preserving the most informative features.

The use of PCA is particularly important in this implementation, as it reduces noise and redundancy, allowing XGBoost to focus on meaningful feature interactions rather than irrelevant variations.

Results and Analysis

XGBoost demonstrates strong performance in detecting fraudulent transactions due to its ability to model complex feature interactions. It achieves high recall, indicating effective identification of fraud cases, while maintaining good precision to avoid excessive false positives.

Compared to other classical models such as Logistic Regression and Random Forest, XGBoost generally provides superior predictive performance. It also serves as a critical benchmark for evaluating quantum models, as any quantum advantage must be measured against such a high-performing classical algorithm.

4.2.1.3 Random Forest

Model Overview

Random Forest is an ensemble-based supervised machine learning algorithm that combines multiple decision trees to improve classification accuracy and robustness. It operates on the principle of "wisdom of the crowd," where multiple weak learners (decision trees) collectively produce a strong predictive model.

In the context of credit card fraud detection, Random Forest is particularly useful because transaction data often contains noise, irrelevant features, and complex interactions. By training multiple trees on randomly sampled subsets of data and features, the model reduces variance and improves generalization. This makes it less prone to overfitting compared to a single decision tree.

In this project, Random Forest is used as a stable and reliable classical model that provides consistent performance. It serves as an important reference point to evaluate how ensemble learning compares with both simpler models (like Logistic Regression) and more advanced models (like XGBoost and quantum approaches).

Mathematical Foundation

Random Forest is based on the aggregation of multiple decision trees. Each tree makes an independent prediction, and the final output is determined using majority voting:

$$\hat{y} = \text{mode}(f_1(x), f_2(x), \ldots, f_n(x))$$

where $f_k(x)$ represents the prediction of the k-th decision tree.

Each decision tree is constructed using recursive binary splitting based on impurity measures such as:

Gini Index: $\text{Gini} = 1 - \Sigma\,(p_i)^2$

$$\text{Entropy} = -\,\Sigma\, p_i \log(p_i)$$

These measures determine how well a feature splits the data into distinct classes.

Working Mechanism

Random Forest builds multiple decision trees independently using bootstrap sampling (sampling with replacement). Each tree is trained on a different subset of the dataset, ensuring diversity among trees.

Additionally, at each split in a tree, only a random subset of features is considered. This randomness reduces correlation between trees and improves overall model performance.

During prediction, each tree provides a classification result (fraud or non-fraud), and the final decision is made based on majority voting. This aggregation helps reduce variance and improves robustness against noise in the data.

In this implementation, Random Forest processes PCA-reduced features and identifies patterns that distinguish fraudulent transactions by analyzing multiple decision paths across trees.

Algorithm

Step 1: Data Preprocessing

Apply feature scaling and PCA transformation to reduce dimensionality. Balance the dataset using SMOTE.

Step 2: Bootstrap Sampling

Generate multiple subsets of the dataset by sampling with replacement. Each subset is used to train a separate decision tree.

Step 3: Feature Selection at Split

At each node in a decision tree, randomly select a subset of features instead of considering all features.

Step 4: Node Splitting

Evaluate possible splits using impurity measures such as Gini index or entropy, and choose the best split.

Step 5: Tree Construction

Repeat splitting process recursively until stopping criteria are met (e.g., maximum depth or minimum samples per leaf).

Step 6: Ensemble Formation

Construct multiple such trees independently using different bootstrap samples.

Step 7: Prediction Phase

Each tree predicts the class label for a given input.

Step 8: Majority Voting

Aggregate predictions from all trees and assign the final class based on majority vote.

Training and Data Handling

Random Forest is trained on a balanced dataset created using SMOTE, ensuring that both fraud and non-fraud classes are equally represented. Feature scaling and PCA transformation ensure that the model receives normalized and noise-reduced input.

Since Random Forest is relatively robust to scaling compared to other models, the primary benefit of preprocessing in this case is dimensionality reduction and noise elimination.

Results and Analysis

Random Forest provides stable and consistent performance across different folds of cross-validation. It effectively reduces overfitting compared to a single decision tree and performs well in capturing general patterns in the data.

However, compared to XGBoost, Random Forest may show slightly lower performance due to the lack of sequential learning. It still serves as a strong baseline ensemble model and provides valuable insight into how classical ensemble methods compare with quantum approaches.

4.2.1.4 Logistic Regression

Model Overview

Logistic Regression is a fundamental supervised machine learning algorithm used for binary classification tasks. Despite its simplicity, it serves as a strong baseline model due to its interpretability and computational efficiency. In the context of credit card fraud detection, it evaluates how effectively a linear model can separate fraudulent and non-fraudulent transactions after preprocessing.

In this project, Logistic Regression is used as a reference model to highlight the limitations of linear decision boundaries on complex datasets. Since fraud patterns are often non-linear, this model helps quantify the performance gap between linear methods and more advanced approaches such as ensemble and quantum models.

Mathematical Foundation

Logistic Regression models the probability of a data point belonging to a class using the sigmoid function:

$$\sigma(z) = 1 / (1 + e^{-z}) \qquad \text{where: } z = w \cdot x + b$$

Here, w represents the weight vector, x is the input feature vector, and b is the bias term. The output of the sigmoid function lies between 0 and 1, representing the probability of a transaction being fraudulent.

The model is trained by minimizing the binary cross-entropy (log-loss):

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

This loss function penalizes incorrect predictions and ensures reliable probability estimation.

Working Mechanism

The model computes a weighted sum of the preprocessed input features (after scaling and PCA) and passes it through the sigmoid function to obtain a probability score.

If the predicted probability exceeds a threshold (typically 0.5), the transaction is classified as fraudulent; otherwise, it is classified as non-fraudulent.

Since Logistic Regression assumes a linear relationship between features and the log-odds of the outcome, its performance depends on how well the data becomes linearly separable after preprocessing. PCA aids this process by projecting data into a lower-dimensional space with improved separability.

Algorithm

Step 1: Data Preprocessing

Apply feature scaling, perform PCA for dimensionality reduction, and balance the dataset using SMOTE.

Step 2: Initialization

Initialize model parameters (weights and bias).

Step 3: Linear Combination

Compute $z = w \cdot x + b$ for each data point.

Step 4: Sigmoid Transformation

Convert z into probability using $\hat{y} = \sigma(z)$.

Step 5: Loss Computation

Calculate binary cross-entropy loss.

Step 6: Gradient Computation

Compute gradients of loss with respect to parameters.

Step 7: Parameter Update

Update weights and bias using gradient descent.

Step 8: Iterative Training

Repeat optimization steps until convergence.

Step 9: Prediction Phase

Use trained model to generate probabilities and classify based on threshold.

Training and Data Handling

The model is trained on a SMOTE-balanced dataset, ensuring equal representation of both classes. Feature scaling is essential due to gradient-based optimization, while PCA reduces dimensionality and removes redundant information.

Results and Analysis

Logistic Regression provides a baseline for comparison. While it performs reasonably well after preprocessing, it is limited in capturing non-linear relationships, leading to lower performance compared to models such as SVM and XGBoost.

It may exhibit lower recall, indicating missed fraud cases. However, its simplicity and interpretability make it valuable for understanding feature influence and benchmarking more complex models.

4.2.2 Quantum Model Baselines

4.2.2.1 Quantum Support Vector Machine (QSVM – Feature Map Variants)

Model Overview

The Quantum Support Vector Machine (QSVM) is a quantum-enhanced classification model that extends the classical SVM by mapping data into a high-dimensional quantum Hilbert space. Instead of computing similarity in classical space, QSVM evaluates inner products using quantum circuits.

In this project, two feature mapping strategies are implemented: the ZZ Feature Map and the Pauli Feature Map. Both approaches encode classical data into quantum states but differ in how feature interactions are represented. This comparison allows analysis of how encoding strategies influence fraud detection performance.

Mathematical Foundation

Classical data x is encoded into a quantum state: $|\varphi(x)\rangle = U(x)|0\rangle$

The quantum kernel is defined as: $K(x, x') = |\langle\varphi(x)|\varphi(x')\rangle|^2$

This kernel measures similarity between quantum states and replaces classical kernel functions such as RBF.

Working Mechanism

The QSVM model encodes input features into quantum states using a feature map, constructs a quantum kernel matrix by evaluating overlaps between states, and then applies a classical SVM classifier.

The difference between the two variants lies in the encoding circuit:

ZZ Feature Map: Uses entanglement-based encoding to capture pairwise feature interactions

Pauli Feature Map: Uses rotations based on Pauli operators to create alternative feature representations

This results in different geometries of the feature space, influencing classification performance.

Algorithm

Step 1: Apply preprocessing (scaling, PCA, SMOTE)

Step 2: Encode data using selected feature map (ZZ or Pauli)

Step 3: Construct quantum circuit with entanglement

Step 4: Compute quantum kernel matrix

Step 5: Train classical SVM using kernel

Step 6: Optimize decision boundary

Step 7: Predict new samples using kernel evaluation

Training and Data Handling

Both variants use identical preprocessing pipelines, including scaling, SMOTE balancing, and PCA reduction. This ensures a fair comparison between encoding strategies.

Scaling ensures compatibility with quantum gates, while PCA reduces dimensionality to match available qubits.

Results and Analysis

Both QSVM variants perform well in low-dimensional feature spaces. The ZZ Feature Map captures stronger entanglement-based interactions, while the Pauli Feature Map provides alternative encoding that may capture different feature relationships.

Performance differences are generally small, indicating that encoding strategy influences results but does not drastically change overall model effectiveness. Neither variant consistently outperforms strong classical models, but both demonstrate the potential of quantum feature mapping.

4.2.2.2 Variational Quantum Classifier (VQC)

Model Overview

The Variational Quantum Classifier (VQC) is a hybrid quantum-classical model that uses parameterized quantum circuits along with classical optimization to perform classification.

Unlike QSVM, which relies on fixed quantum kernels, VQC introduces trainable parameters, allowing the model to learn decision boundaries directly from data.

In this project, VQC is implemented using a TwoLocal ansatz with 16 trainable parameters and optimized using the COBYLA algorithm. This enables the model to adapt to complex fraud patterns, making it one of the most flexible quantum models in the proposed framework.

Mathematical Foundation

Classical data x is encoded into a quantum state:

$$|\varphi(x)\rangle = U(x)|0\rangle$$

A parameterized circuit is then applied:

$$|\psi(x, \theta)\rangle = U(\theta)\, U(x)\, |0\rangle$$

where θ represents trainable parameters. The output is obtained using expectation values:

$$\hat{y} = \langle\psi(x, \theta)|\, M\, |\psi(x, \theta)\rangle$$

The model is trained by minimizing a loss function using classical optimization.

Working Mechanism

The VQC model encodes classical features into quantum states and processes them through a parameterized quantum circuit. The circuit outputs are measured and interpreted as prediction probabilities.

A classical optimizer iteratively updates the circuit parameters to minimize the loss between predicted and actual labels. This hybrid training loop continues until convergence, allowing the model to learn adaptive decision boundaries.

The use of a shallow circuit ensures compatibility with NISQ devices while maintaining reasonable expressiveness.

Algorithm

Step 1: Apply preprocessing (scaling, PCA, SMOTE)

Step 2: Encode data into quantum states

Step 3: Initialize parameterized circuit (TwoLocal ansatz)

Step 4: Perform forward pass through quantum circuit

Step 5: Compute loss between predictions and labels

Step 6: Update parameters using COBYLA optimizer

Step 7: Repeat training until convergence

Step 8: Use optimized circuit for prediction

Training and Data Handling

The model follows the same preprocessing pipeline as other models, ensuring fair comparison. Data is scaled and reduced using PCA to match quantum circuit constraints.

The hybrid training process involves repeated quantum circuit evaluation and classical optimization,

Results and Analysis

VQC demonstrates the ability to learn adaptive decision boundaries and capture non-linear patterns. Compared to QSVM, it offers greater flexibility due to trainable parameters.

However, performance depends on optimization efficiency and parameter initialization. In some cases, convergence challenges may limit performance.

4.2.2.3 Quantum Neural Network (QNN)

Model Overview

The Quantum Neural Network (QNN) is a quantum counterpart of classical neural networks that uses parameterized quantum circuits to model complex data patterns. Instead of classical weights and activations, QNN employs quantum gates and measurements to learn feature relationships.

In this project, the QNN is implemented using a RealAmplitudes ansatz with 8 trainable parameters and shallow circuit depth, ensuring compatibility with NISQ devices. The model is included to evaluate whether quantum neural architectures can effectively learn non-linear fraud patterns.

Mathematical Foundation

Classical data x is encoded into a quantum state: $|\varphi(x)\rangle = U(x)|0\rangle$

A parameterized circuit is applied: $|\psi(x, \theta)\rangle = U(\theta)\, U(x)\, |0\rangle$

The output is obtained as expectation values: $\hat{y} = \langle\psi(x, \theta)|\, M\, |\psi(x, \theta)\rangle$

These values are interpreted as prediction probabilities and optimized using a classical loss function.

Working Mechanism

The QNN operates in a hybrid manner. Input features are encoded into quantum states, followed by a parameterized circuit that applies rotations and entanglement across qubits.

The circuit produces measurement outputs, which are compared with true labels to compute loss. A classical optimizer updates the circuit parameters iteratively, enabling the model to learn patterns from the data.

The shallow circuit design ensures efficiency but may limit representational capacity.

Algorithm:

Step 1: Apply preprocessing (scaling, PCA, SMOTE)

Step 2: Encode data using quantum rotation gates

Step 3: Initialize parameterized circuit (RealAmplitudes ansatz)

Step 4: Perform forward pass and measure outputs

Step 5: Compute loss between predictions and labels

Step 6: Update parameters using classical optimizer

Step 7: Repeat until convergence

Step 8: Use trained circuit for prediction

Training and Data Handling

The QNN uses the same preprocessing pipeline as other models, including scaling, SMOTE balancing, and PCA reduction. This ensures consistency and fair comparison.

The hybrid training process involves repeated quantum evaluations and classical optimization steps.

Results and Analysis

The QNN shows moderate performance in fraud detection tasks. While capable of learning patterns, its effectiveness is limited by shallow circuit depth and fewer trainable parameters compared to VQC.

The results highlight both the potential and current limitations of quantum neural architectures in practical applications.

4.2.2.4 Quantum Kernel Alignment (QKA)

Model Overview

Quantum Kernel Alignment (QKA) is an advanced quantum machine learning approach that extends traditional quantum kernel methods by introducing trainable parameters into the feature map. Unlike fixed-kernel models such as QSVM, QKA adapts the kernel to the underlying data, improving its ability to separate classes.

In this project, QKA represents the key contribution of the proposed framework, as it combines kernel-based learning with variational optimization. By aligning the quantum kernel with target labels, the model aims to enhance fraud detection performance, especially in complex and non-linear scenarios.

Mathematical Foundation

Classical data x is encoded into a parameterized quantum state: $|\varphi(x, \theta)\rangle = U(x, \theta)|0\rangle$

The quantum kernel is defined as: $K(x, x'; \theta) = |\langle\varphi(x, \theta)|\varphi(x', \theta)\rangle|^2$

Kernel alignment is computed as: $A(K, Y) = \langle K, YY^T\rangle / (\|K\| \, \|YY^T\|)$

The objective is to optimize parameters θ to maximize alignment, improving class separability in quantum space.

Working Mechanism

The QKA model encodes data using a parameterized quantum feature map and computes a quantum kernel matrix. Unlike QSVM, the feature map parameters are iteratively updated using a classical optimizer to maximize kernel alignment with labels.

At each iteration, the kernel matrix is evaluated, and alignment is computed. The optimizer adjusts parameters to improve alignment, effectively reshaping the feature space.

Once optimized, the final kernel is used to train a classical SVM for classification.

Algorithm

Step 1: Apply preprocessing (scaling, PCA, SMOTE)

Step 2: Initialize trainable parameters θ

Step 3: Encode data using parameterized feature map

Step 4: Compute quantum kernel matrix

Step 5: Calculate kernel alignment with labels

Step 6: Define loss based on alignment

Step 7: Update parameters using classical optimizer

Step 8: Repeat until convergence

Step 9: Compute optimized kernel

Step 10: Train classical SVM using optimized kernel

Training and Data Handling

The model follows the same preprocessing pipeline as other models to ensure consistency. Scaling and PCA are essential for quantum compatibility, while SMOTE ensures balanced class representation.

The hybrid training process involves repeated kernel evaluation and parameter updates.

Evaluation Metrics

Model performance is evaluated using Precision, Recall, and F1 Score to measure prediction accuracy and balance, while ROC-AUC and PR-AUC assess classification effectiveness, especially on imbalanced fraud data.

Results and Analysis

QKA demonstrates improved performance compared to fixed quantum kernel models such as QSVM. By adapting the feature map, it achieves better alignment between data representation and class labels.

In certain cases, it shows competitive performance with classical models, indicating potential quantum advantage under specific conditions. However, performance depends on optimization quality and parameter initialization.

4.3 Integration Details

The system is designed with a unified architecture that integrates classical and quantum models within a single pipeline.

- A shared preprocessing module generates a common dataset used across all models.
- A unified evaluation module computes metrics such as Precision, Recall, F1 Score, ROC-AUC, and PR-AUC.
- Cross-validation is applied consistently across all models to ensure fair comparison.

Classical models are implemented using Scikit-learn and XGBoost libraries, while quantum models are implemented using Qiskit and executed on Aer simulators. Optimization routines for quantum models are handled using SciPy.

This integration ensures modularity, scalability, and reproducibility of the system.

4.4 Testing

The system is evaluated using a structured testing framework designed to assess performance under different conditions.

The evaluation metrics used include Precision, Recall, F1 Score, ROC-AUC, and PR-AUC, which are particularly suitable for imbalanced classification problems. These metrics provide a comprehensive understanding of model performance beyond simple accuracy.

4.4.1 Test Cases

The following test cases are used to evaluate system performance:

Test Case 1: Standard Model Evaluation

All models are evaluated using the balanced dataset with five-fold cross-validation.

Test Case 2: QABD Analysis

Performance is analysed across different sample sizes (50, 100, 150, 200) and feature dimensions (2, 3, 4).

Test Case 3: Noise Robustness Testing

Quantum models are evaluated under varying noise levels (0% to 15%).

Test Case 4: Extreme Imbalance Testing

Models are tested under highly imbalanced conditions (~0.4% fraud cases).

Test Case 5: Feature Importance Analysis

SHAP is used to evaluate the contribution of PCA features.

CHAPTER 5

DISCUSSION OF RESULTS

This chapter presents the performance evaluation and analysis of the proposed comparative quantum-classical machine learning framework for credit card fraud detection. All models are tested under identical conditions using a shared dataset to ensure fairness. The results are analysed using multiple performance metrics and experimental studies.

Evaluation Metrics

a) **Accuracy**

Accuracy measures the proportion of correctly classified instances out of all instances. It is the most common metric for classification tasks.

$$\textbf{Accuracy}=(TP+TN)/(TP+TN+FP+FN)$$

where:

TP=True Positives (correctly predicted positives)

TN=True Negatives (correctly predicted negatives)

FP=False Positives (incorrectly predicted positives)

FN=False Negatives (incorrectly predicted negatives)

b) **Precision:**

Precision is the proportion of true positives out of all positive predictions.

$$\textbf{Precision}=TP/(TP+FP)$$

c) **Recall (Sensitivity):**

Recall measures the proportion of true positives that were correctly identified by the model out of all actual positives.

$$\textbf{Recall}=TP/(TP+FN)$$

d) **F1 – Score:**

F1-Score is the harmonic mean of precision and recall. It provides a single metric that combines both precision and recall**.**

$$\textbf{F1}=(2\ \ \textbf{Precision}\ \ \textbf{Recall})/(\textbf{Precision}+\textbf{Recall})$$

e) **Confusion Matrix:**

A confusion matrix is a table used in classification to summarize the performance of a classification algorithm. It contains four metrics that are essential for the calculation of other metrics. The confusion matrix structure is shown in the table 5.

Table 5: Confusion Matrix Structure

	Predicted Positive	Predicted Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

f) ROC AUC Score:

ROC(Receiver Operating Characteristic) curve is a graphical plot that illustrates the diagnostic ability of a binary classifier. AUC (Area under the Curve) ROC is a performance measurement for classification problems. The ROC AUC Score is the area under the ROC Curve.

ROC AUC Score=(True Positive Rate False Positive Rate)/2

5.1 Model Performance Analysis

The performance of all classical and quantum models is evaluated using Precision, Recall, F1 Score, ROC-AUC, and PR-AUC metrics. Five-fold stratified cross-validation is applied to ensure consistent evaluation.

Table 5.1: Comparative Performance of Classical and Quantum Models

Model	Type	Recall	Precision	F1	ROC-AUC	PR-AUC	Accuracy	Train Time (s)
SVM (RBF)	Classical	0.8878	1.0000	0.9405	0.9837	0.9858	0.9439	0.0
XGBoost	Classical	0.9082	0.9570	0.9319	0.9681	0.9769	0.9337	0.1
Random Forest	Classical	0.9082	0.9674	0.9368	0.9803	0.9840	0.9388	0.2

Logistic Regression	Classical	0.8776	1.0000	0.9348	0.9807	0.9858	0.9388	0.0
QSVM (ZZ)	Quantum	0.8878	0.9560	0.9206	0.9722	0.9775	0.9235	118.6
QSVM (Pauli)	Quantum	0.8878	0.9560	0.9206	0.9722	0.9775	0.9235	160.5
VQC	Quantum	0.5510	0.9818	0.7059	0.9681	0.9673	0.7704	167.4
QNN	Quantum	0.6020	0.9672	0.7421	0.9146	0.9396	0.7908	166.5
QKA	Quantum	0.8776	1.0000	0.9348	0.9833	0.9863	0.9388	2033.2

From the results, classical models such as XGBoost and Random Forest show strong and stable performance. Among quantum models, QSVM and QKA demonstrate competitive results under constrained feature settings. However, quantum models generally exhibit higher variance due to limited dataset size and circuit depth.

5.2 Experimental Results and Analysis

5.2.1 QABD Analysis (Quantum Advantage Study)

The QABD heatmap shows the F1 score comparison between QSVM-ZZ and classical RBF- SVM across different sample sizes and feature dimensions. It is observed that classical models outperform quantum models in most configurations. A slight quantum advantage is observed in one case (200 samples, 3 features) with a small ΔF1 value. This indicates that quantum models are competitive only under limited conditions.

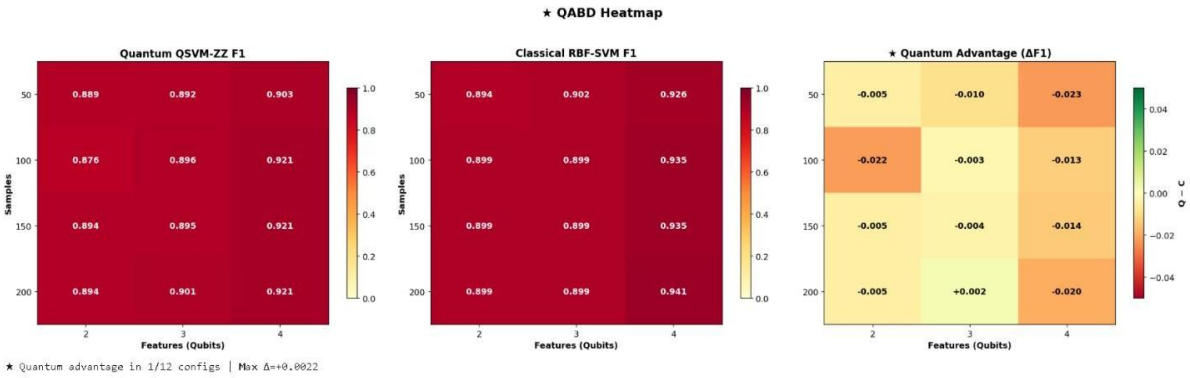

Fig 5.2.1: QABD Heatmap for Quantum and Classical Model Comparison

5.2.2 Noise Robustness Analysis

To evaluate the practical feasibility of quantum models, depolarizing noise is simulated at different levels.

Table 5.2.2: Effect of Noise on Quantum Model Performance

Noise Level	F1 Score
0%	0.9206
1%	0.8947
3%	0.8526
5%	0.7579
10%	0.6096
15%	0.4817

The results show that quantum model performance degrades significantly beyond approximately 5% noise. This highlights the limitations of current NISQ devices and the sensitivity of quantum models to noise.

5.2.3 Feature Importance Analysis

Feature importance is analysed using SHAP on the XGBoost model to understand the contribution of each PCA component. The analysis shows that certain principal components contribute more significantly to fraud detection, indicating that PCA effectively captures relevant information from the original feature space.

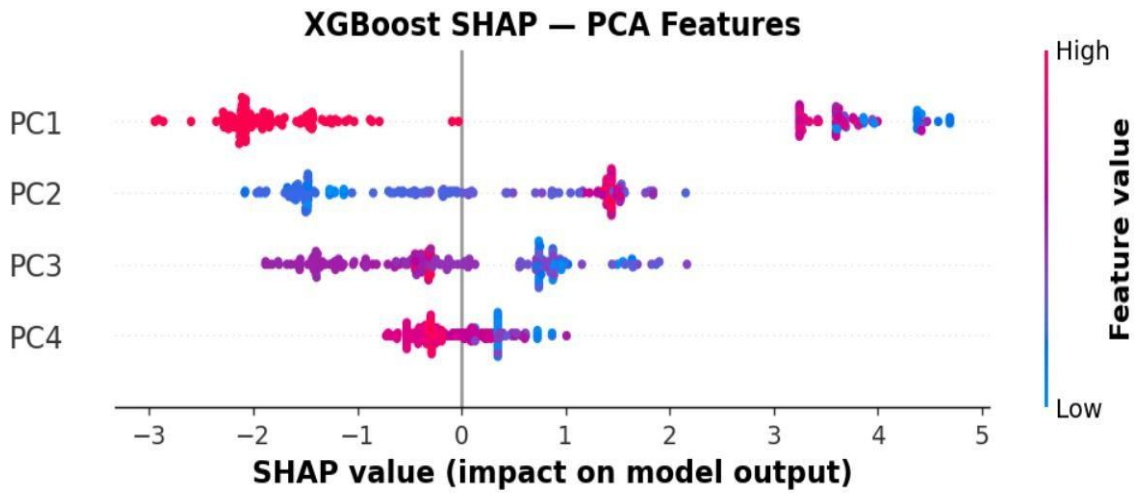

Fig 5.2.3: SHAP Feature Importance Plot

The analysis shows that certain principal components contribute more significantly to fraud

detection, indicating that PCA effectively captures relevant information from the original feature space.

5.2.4 Quantum Kernel PCA Visualization

Quantum Kernel PCA is used to visualize data separation in quantum feature space.

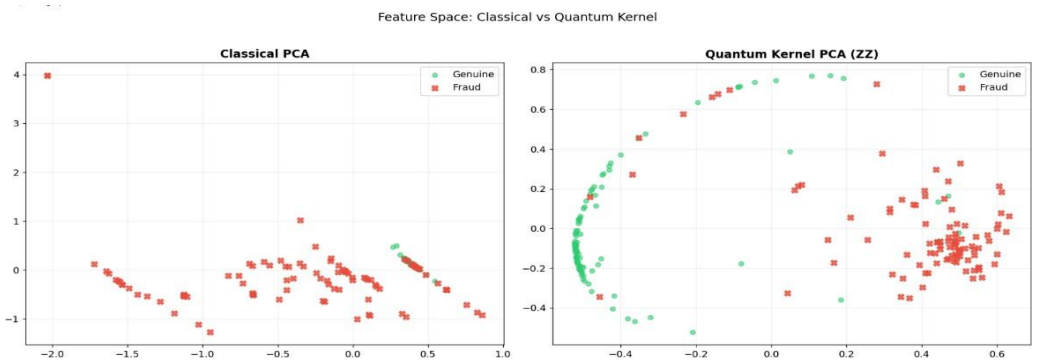

Fig 5.2.4 : Quantum Kernel PCA Projection

5.2.5 Extreme Imbalance Testing

An additional experiment is conducted with extreme class imbalance (~0.4% fraud cases) to evaluate model robustness.

The results show that classical models maintain relatively stable performance, while quantum models experience noticeable degradation. This highlights the challenges of applying quantum models to highly imbalanced real-world datasets.

Table 5.2.5: Model Performance under Extreme Rarity (0.4%)

Model	Recall	F1 Score
SVM (RBF)	0.9	0.9000
XGBoost	0.9	0.8372
Random Forest	0.9	0.8780
Logistic Regression	0.9	0.9474
QSVM (ZZ)	0.9	0.8182
QSVM (Pauli)	0.9	0.8182

QKA	0.9	0.9474
VQC	0.6	0.7059
QNN	0.7	0.7778

5.3 Comparative Analysis with Existing Approches

The performance of the proposed system is compared with existing studies in literature.

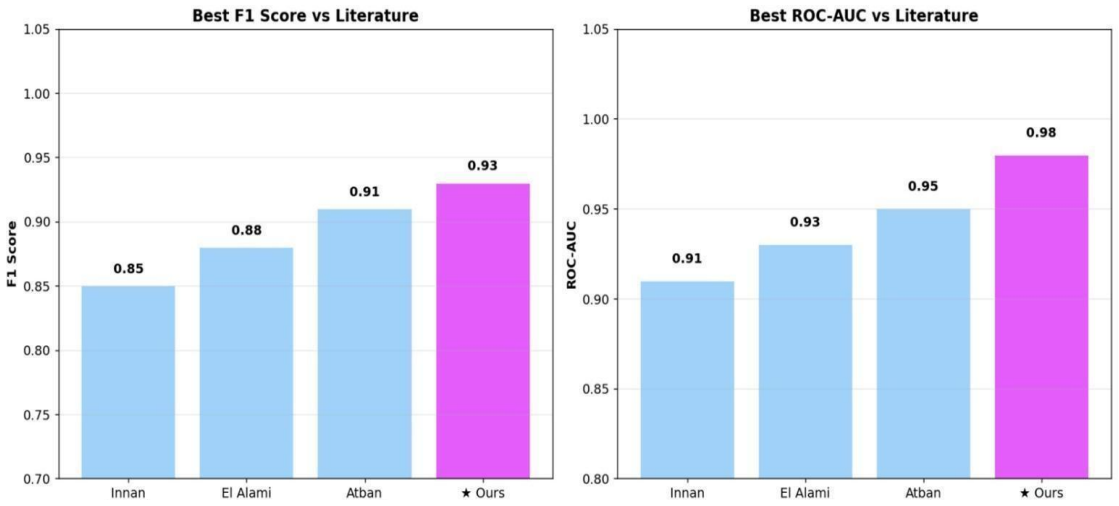

Fig 5.3 : Comparison with Existing Methods

The proposed framework provides a more controlled and fair comparison between classical and quantum models, along with additional analysis such as QABD and noise robustness, which are not commonly addressed in prior work.

Summary of Results

Overall, classical models outperform quantum models in most scenarios, especially with larger datasets. However, quantum models show competitive performance under constrained conditions and provide insights into alternative feature representations.

The study highlights both the potential and limitations of quantum machine learning for financial fraud detection, emphasizing the need for further advancements in quantum hardware and algorithms.

CHAPTER 6

CONCLUSION & FUTURE SCOPE

6.1 Conclusion

This project presents a comparative study of classical and quantum machine learning models for credit card fraud detection under a controlled and fair experimental setup. A shared preprocessing pipeline is used to generate a balanced dataset of 200 samples with four PCA-reduced features, ensuring identical training conditions for all models.

The experimental results show that classical models such as XGBoost, Random Forest, and SVM consistently achieve strong and stable performance across all evaluation metrics. Quantum models, including QSVM, VQC, QNN, and QKA, demonstrate competitive performance in certain configurations but do not consistently outperform classical approaches. The QABD analysis indicates that quantum advantage is limited and occurs only under specific conditions, particularly with lower feature dimensions and moderate sample sizes. Noise analysis further reveals that quantum models are sensitive to noise, with noticeable performance degradation beyond approximately 5% noise levels. Overall, the study highlights that while quantum machine learning shows potential, classical models remain more reliable and practical for real-world fraud detection Tasks under current technological constraints.

6.2 Future Scope

While the current model offers promising results, there is significant potential for future enhancements that could improve accuracy, scalability, and real-world applicability:

1. **Use of Larger and Real-World Datasets**

Expanding the study to larger and more diverse financial datasets can provide a better evaluation of model scalability and robustness. This would help in validating the performance of both classical and quantum models in real-world fraud detection scenarios.

2. **Hybrid Quantum-Classical Models**

Future work can explore hybrid approaches where classical and quantum components are integrated within a single model. This may improve performance by combining the strengths of both paradigms.

3. **Advanced Quantum Circuit Design**

Developing deeper and more expressive quantum circuits with improved feature

encoding techniques could enhance the learning capability of quantum models and potentially improve their performance.

4. Evaluation on Real Quantum Hardware

The current study relies on quantum simulators. Future research can implement these models on real quantum devices to better understand practical limitations such as noise and hardware constraints.

5. Noise Reduction and Error Mitigation

Improving quantum error mitigation techniques can help reduce the impact of noise, thereby enhancing the reliability and stability of quantum model predictions.

6. Explainable AI Techniques

Incorporating explainable AI methods such as SHAP or LIME can improve model interpretability, allowing better understanding of decision-making in fraud detection systems.

REFERENCES

[1] M. Schuld and N. Killoran, "Quantum machine learning in feature Hilbert spaces," *Phys. Rev. Lett.*, vol. 122, no. 4, 2019.

[2] V. Havlíček et al., "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, pp. 209–212, 2019.

[3] D. Alvarez-Estevez, "Benchmarking quantum machine learning kernel training for classification tasks," *arXiv preprint arXiv:2408.10274*, 2024.

[4] J. R. Glick et al., "Covariant quantum kernels for data with group structure," *Nat. Phys.*, vol. 20, pp. 479–483, 2024.

[5] N. Innan, M. A. Khan, and M. Bennai, "Financial fraud detection: a comparative study of quantum machine learning models," *Int. J. Quantum Inf.*, vol. 22, no. 2, 2024.

[6] M. El Alami et al., "Comparative performance analysis of quantum ML architectures for credit card fraud detection," *arXiv preprint arXiv:2412.19441*, 2024.

[7] F. Atban, M. Y. Küçükkara, and C. Bayılmış, "Enhancing variational quantum classifier performance with meta-heuristic feature selection," *Eur. Phys. J. Spec. Top.*, vol. 234, pp. 3705–3718, 2025.

[8] C. Huot, S. Heng, T.-K. Kim, and Y. Han, "Quantum autoencoder for enhanced fraud detection in imbalanced credit card dataset," *IEEE Access*, 2024.

[9] N. Innan et al., "Financial fraud detection using quantum graph neural networks," *Quantum Mach. Intell.*, vol. 6, no. 1, p. 7, 2024.

[10] J. M. Neto and G. P. Temporão, "Fraud detection in credit card transactions using quantum-assisted restricted Boltzmann machines," *arXiv preprint arXiv:2512.17660*, 2024.

[11] T. Hubregtsen et al., "Training quantum embedding kernels on near-term quantum computers," Phys. Rev. A, vol. 106, no. 4, 2022.

[12] A. Di Pierro and M. Incudini, "Implementing quantum machine learning in credit card fraud detection," in Lecture Notes in Computer Science (LNCS), vol. 676, Springer, 2021.

[13] F. K. Alarfaj et al., "Credit card fraud detection using state-of-the-art machine learning anddeep learning algorithms," IEEE Access, vol. 10, pp. 39700–39715, 2022.

[14] I. D. Mienye and N. Jere, "Deep learning for credit card fraud detection: A review of algorithms, challenges, and solutions," IEEE Access, 2024.

[15] H. Fanai and H. Abbasimehr, "A novel combined approach based on deep autoencoder and deep classifiers for credit card fraud detection," Expert Syst. Appl., vol. 217, p. 119562, 2023.

[16] G. Zioviris, K. Kolomvatsos, and G. Stamoulis, "Credit card fraud detection using a deep

learning multistage model," J. Supercomput., vol. 78, pp. 14571–14596, 2022.

[17] S. K. Hashemi et al., "Fraud detection with ensemble methods and SMOTE oversampling," Appl. Sci., 2023.

[18] P. Kumar et al., "Credit card fraud detection using machine learning: A systematic review," J. Comput. Sci., vol. 44, 2020.

[19] R. Shaydulin and S. M. Wild, "Importance of kernel bandwidth in quantum machine learning," Phys. Rev. A, vol. 106, 2022.

[20] N. Mohanty, T. Vuppala, and P. Kulkarni, "Implementing quantum machine learning in credit card fraud detection," in Proc. IEEE Conf., 2024.

[21]S. Ramos Pulido, N. Hernández-Gress, and G. Uehara, "Hybrid quantum-classical neural networks for fraud classification," in Proc. IEEE Conf., 2025.

APPENDIX

Sample Code

This appendix presents selected code snippets from the implementation of the comparative quantum-classical fraud detection framework.

PACKAGE INSTALLATION

```
import subprocess, sys
packages = ['qiskit==1.2.4', 'qiskit-
    aer==0.15.1', 'qiskit-machine-
    learning==0.7.2',
    'qiskit-
    algorithms==0.3.1',
    'imbalanced-learn',
    'xgboost', 'shap']
for pkg in packages:
  subprocess.check_call([sys.executable, '-m',
  'pip', 'install',
            '-q', pkg],
            stdout=subprocess.DEVNUL
            L,
            stderr=subprocess.DEVNUL
            L)
```

IMPORTS AND CONFIGURATION

```
import warnings;
warnings.filterwarnings('ignore') import
numpy as np, pandas as pd
```

```
import matplotlib.pyplot as plt, seaborn as sns, time
from sklearn.model_selection import train_test_split,
StratifiedKFold from sklearn.preprocessing import
StandardScaler, MinMaxScaler from sklearn.decomposition
import PCA
from sklearn.metrics import (classification_report,
  confusion_matrix, roc_auc_score, roc_curve,
  precision_recall_curve, auc,
 recall_score, precision_score, f1_score,
accuracy_score) from sklearn.svm import SVC
from sklearn.ensemble import
RandomForestClassifier from
sklearn.linear_model import
LogisticRegression from xgboost import
XGBClassifier
from imblearn.over_sampling import SMOTE
from scipy.optimize import minimize as
scipy_minimize import
qiskit_machine_learning
qml_v = qiskit_machine_learning.__version__
USE_OLD_API = tuple(int(x) for x in qml_v.split('.')[:2]) < (0, 7)
SEED = 42
np.random.seed(SEED)
```

DATASET LOADING AND EXPLORATION

```
df = pd.read_csv("creditcard.csv")
print(f"Shape: {df.shape} | Fraud: {df['Class'].sum()} "
  f"({df['Class'].mean()  100:.4f}%) | Missing: "
```

```
f"{df.isnull().sum().sum()}")
```

PREPROCESSING PIPELINE (6 Stages)

```
X =
df.drop('Class',
axis=1) y =
df['Class']
    Stage 1 — Stratified 80/20 split
X_train_full, X_test_full, y_train_full, y_test_full =
  train_test_split( X, y, test_size=0.2,
  random_state=SEED, stratify=y)
    Stage 2 — Standard
scaling scaler =
StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train_full) X_test_scaled
= scaler.transform(X_test_full)
    Stage 3 — SMOTE oversampling (training partition
only) smote = SMOTE(random_state=SEED)
X_train_balanced, y_train_balanced = smote.fit_resample(
  X_train_scaled, y_train_full)
    Stage 4 — PCA: 30 features → 4 components (86% variance retained)
NUM_QUBITS   = 4
Q_TRAIN_SIZE = 200
pca = PCA(n_components=NUM_QUBITS,
random_state=SEED) X_train_pca =
pca.fit_transform(X_train_balanced)
X_test_pca  = pca.transform(X_test_scaled)
```

```
print(f"PCA variance retained: {pca.explained_variance_ratio_.sum()
100:.1f}%")    Stage 5 — Balanced subsample: 100 fraud + 100 genuine
fraud_idx  = np.where(y_train_balanced ==
1)[0] genuine_idx = np.where(y_train_balanced
== 0)[0] np.random.seed(SEED)
selected_train = np.concatenate([
  np.random.choice(fraud_idx,  Q_TRAIN_SIZE // 2, replace=False),
np.random.choice(genuine_idx, Q_TRAIN_SIZE // 2, replace=False)])
X_train_shared = X_train_pca[selected_train]
y_train_shared =
pd.Series(y_train_balanced).iloc[selected_train].values
Stage 6 — MinMax scaling to [-1, 1] for quantum encoding
scaler_q  =
MinMaxScaler(feature_range=(-1, 1))
X_train_q  =
scaler_q.fit_transform(X_train_shared)
X_test_q  =
scaler_q.transform(X_test_pca)
  Balanced test set: 98 fraud + 98 genuine
fraud_test_idx  = np.where(y_test_full ==
1)[0] genuine_test_idx =
np.where(y_test_full == 0)[0]
np.random.seed(SEED)
selected_test = np.concatenate([fraud_test_idx,
np.random.choice(genuine_test_idx, len(fraud_test_idx),
replace=False)]) X_test_shared = X_test_q[selected_test]
y_test_shared = y_test_full.iloc[selected_test]
```

```
print(f"Shared train: {X_train_q.shape} | Shared test:
{X_test_shared.shape}") print("Both classical AND quantum
models train on identical 200 samples
    scores[name]    = y_score
```

CLASSICAL MODEL TRAINING

```
classical_configs = {
  'SVM (RBF)': SVC(kernel='rbf',
        gamma='scale', probability=True,
        random_state=SEED),
  'XGBoost':  XGBClassifier(n_estimators=100,
            max_depth=4, learning_rate=0.1,
            random_state=SEED, use_label_encoder=False,
            eval_metric='logloss'),
  'Random Forest':
                    RandomForestClassifier(n_estim
          ators=100, max_depth=8,
          random_state=SEED),
  'Logistic Regression': LogisticRegression(max_iter=1000,
            random_state=SEED),
}

for name, model in
  classical_configs.items(): t0 =
  time.time()
  model.fit(X_train_q,
  y_train_shared) timings[name] =
```

```
        time.time() - t0
        yp   = model.predict(X_test_shared)
        yproba =
        model.predict_proba(X_test_shared)[:, 1]
        evaluate_model(name, y_test_shared, yp,
        yproba) print(f"{name} —
        {timings[name]:.2f}s")
  print(classification_report(y_test_shared, yp, digits=4))
```

QUANTUM BACKEND SETUP

```
from qiskit.circuit.library import (ZZFeatureMap,
            PauliFeatureMap, TwoLocal, RealAmplitudes)
from qiskit.circuit import
ParameterVector from qiskit
import QuantumCircuit
from qiskit.quantum_info import
Statevector if USE_OLD_API:
  from qiskit_aer import Aer
  from qiskit.utils import QuantumInstance
  from qiskit_machine_learning.kernels import
  QuantumKernel backend    =
  Aer.get_backend('aer_simulator')
  quantum_instance = QuantumInstance(backend,
  shots=256,
              seed_simulator=SEED,
              seed_transpiler=SEED)
else:
  from qiskit_machine_learning.kernels import
```

FidelityQuantumKernel print("Quantum backend ready.")

QSVM — ZZFeatureMap

feature_map_zz =

ZZFeatureMap(feature_dimension=NUM_QUBITS,

reps=1, entanglement='linear')

qkernel_zz = (QuantumKernel(feature_map=feature_map_zz,

quantum_instance=quantum_instance)

if USE_OLD_API else
FidelityQuantumKernel(feature_map=feature_map_

zz)) qsvm_zz = SVC(kernel=qkernel_zz.evaluate)

t0 = time.time()

qsvm_zz.fit(X_train_q,

y_train_shared)

timings['QSVM (ZZ)'] =

time.time() - t0 yp =

qsvm_zz.predict(X_test_shared

)

ysc =

qsvm_zz.decision_function(X_test_shared)

evaluate_model('QSVM (ZZ)',

y_test_shared, yp, ysc) print(f"QSVM (ZZ)

— {timings['QSVM (ZZ)']:.1f}s")

print(classification_report(y_test_shared, yp,

digits=4)) **QSVM — PauliFeatureMap**

feature_map_pauli =

PauliFeatureMap(feature_dimension=NUM_QUBITS,

reps=1, paulis=['Z', 'ZZ'],

```
                entanglement='linear')
        qkernel_pauli =
            (QuantumKernel(feature_map=feature_map_pauli,
            quantum_instance=quantum_instance)
            if USE_OLD_API else
            FidelityQuantumKernel(feature_map=feature_m
            ap_pauli))
        qsvm_pauli =
        SVC(kernel=qkernel_pauli.evaluate)  t0
        = time.time()
        qsvm_pauli.fit(X_train_q,
        y_train_shared) timings['QSVM
        (Pauli)'] = time.time() - t0 yp =
        qsvm_pauli.predict(X_test_shar
        ed)
        ysc = qsvm_pauli.decision_function(X_test_shared)
        evaluate_model('QSVM (Pauli)', y_test_shared,
        yp, ysc) print(f"QSVM (Pauli) —
        {timings['QSVM (Pauli)']:.1f}s")
print(classification_report(y_test_shared, yp,
digits=4))
```

VQC — Variational Quantum Classifier

```
vqc_circuit =
QuantumCircuit(NUM_QUBITS)
vqc_circuit.compose(feature_map_zz,
inplace=True) ansatz_vqc =
TwoLocal(num_qubits=NUM_QUBITS,
        rotation_blocks=['r
```

```
            y', 'rz'],
            entanglement_block
            s='cz', reps=1,
            entanglement='line
            ar')
vqc_circuit.compose(ansatz_vqc,
inplace=True) fm_params    =
list(feature_map_zz.parameters)
vqc_ans_params =
list(ansatz_vqc.parameters) def
parity_proba(probs):
  return sum(probs[k] for k in
        range(len(probs)) if
        bin(k).count('1') % 2 == 1)
def vqc_forward(X, weights):
  p  = np.zeros(len(X))
  wd = {par: w for par, w in zip(vqc_ans_params,
  weights)} for i in range(len(X)):
    fd = {  wd,   {par: X[i,j] for j,par in
    enumerate(fm_params)}} sv =
    Statevector.from_instruction(
       vqc_circuit.assign_paramet
  ers(fd)) p[i] =
  parity_proba(sv.probabilities(
  )) return p
def vqc_loss(w, X, y):
  p = np.clip(vqc_forward(X, w), 1e-10, 1 -
```

```
1e-10) return -np.mean(y    np.log(p) + (1 -
y)    np.log(1 - p))
      np.random.see
      d(SEED) t0 =
      time.time()
      vqc_result = scipy_minimize(vqc_loss,
        np.random.uniform(-np.pi, np.pi,
        len(vqc_ans_params)), args=(X_train_q,
        y_train_shared),
        method='COBYLA',
      options={'maxiter': 100})
      timings['VQC'] = time.time() - t0
      vp       = vqc_forward(X_test_shared,
      vqc_result.x) y_pred_vqc  = (vp >=
      0.5).astype(int) evaluate_model('VQC',
      y_test_shared, y_pred_vqc, vp)
      print(f"VQC — {timings['VQC']:.1f}s | loss:
      {vqc_result.fun:.4f}")
      print(classification_report(y_test_shared.values,
      y_pred_vqc,
                digits=4, zero_division=0))
```

QNN — Quantum Neural Network

```
      qnn_ansatz =
                RealAmplitudes(num_qubits=NUM_QUBITS,
                reps=1, entanglement='linear')
      qnn_circuit =
      QuantumCircuit(NUM_QUBITS)
```

```
qnn_circuit.compose(feature_map_zz,
inplace=True)
qnn_circuit.compose(qnn_ansatz,
inplace=True) qnn_ans_params =
list(qnn_ansatz.parameters) print(f"QNN
— depth: {qnn_circuit.depth()}, "
   f"params:
{len(qnn_ans_params)}")
def qnn_forward(X,
weights):
  p  = np.zeros(len(X))
  wd = {par: w for par, w in zip(qnn_ans_params,
  weights)} for i in range(len(X)):
  fd = {  wd,   {par: X[i,j] for j,par in
  enumerate(fm_params)}} sv =
  Statevector.from_instruction(
    qnn_circuit.assign_parame
 ters(fd)) p[i] =
 parity_proba(sv.probabilities(
 )) return p
def qnn_loss(w, X, y):
 p = np.clip(qnn_forward(X, w), 1e-10,
1 - 1e-10) return np.mean((p - y)    2)
np.random.seed(SEED)
t0 = time.time()
qnn_result = scipy_minimize(qnn_loss,
 np.random.uniform(-np.pi, np.pi,
```

len(qnn_ans_params)), args=(X_train_q,

y_train_shared),

method='COBYLA', options={'maxiter':

100}) timings['QNN'] = time.time() - t0

qp = qnn_forward(X_test_shared,

qnn_result.x) y_pred_qnn = (qp >=

0.5).astype(int) evaluate_model('QNN',

y_test_shared, y_pred_qnn, qp)

print(f"QNN — {timings['QNN']:.1f}s | loss:

{qnn_result.fun:.4f}")

print(classification_report(y_test_shared.values, y_pred_qnn,

digits=4, zero_division=0))

QKA — Quantum Kernel Alignment (Novel

Contribution) data_params = ParameterVector('x',

NUM_QUBITS) train_params = ParameterVector('t',

NUM_QUBITS 2)

def build_qka_circuit(tv):

qc = QuantumCircuit(NUM_QUBITS)

for i in range(NUM_QUBITS):

qc.ry(data_params[i], i)

for i in range(NUM_QUBITS - 1):

qc.cx(i, i + 1)

for i in range(NUM_QUBITS):

qc.ry(train_params[i], i)

qc.rz(train_params[NUM_QUBI

TS + i], i) for i in

range(NUM_QUBITS - 1):

```
        qc.cx(i, i + 1)
      for i in range(NUM_QUBITS):
        qc.rz(data_params[i], i)
      return qc.assign_parameters(
        {train_params[i]: tv[i] for i in range(NUM_QUBITS
    2)}) best_qka_score = -1
    best_qka_model = None
    print("Training QKA —
    3 trials...") t0 =
    time.time()
    for trial in range(3):
      np.random.seed(SE
      ED + trial) bfm =
      build_qka_circuit(
        np.random.uniform(-np.pi, np.pi,
      NUM_QUBITS    2)) qk  =
      (QuantumKernel(feature_map=bfm,
          quantum_instance=quantum
          _instance) if
          USE_OLD_API else
          FidelityQuantumKernel(feature_map=bfm))
      svc = SVC(kernel=qk.evaluate)
svc.fit(X_train_q, y_train_shared)
f1  = f1_score(y_test_shared,
      svc.predict(X_test_shared), zero_division=0)
print(f"  Trial {trial+1}: F1 =
{f1:.4f}") if f1 >
```

```
  best_qka_score:
    best_qka_score = f1
    best_qka_model = svc
timings['QKA'] = time.time() - t0
yp  = best_qka_model.predict(X_test_shared)
ysc =
best_qka_model.decision_function(X_test_shar
ed) evaluate_model('QKA', y_test_shared, yp,
ysc) print(f"QKA — {timings['QKA']:.1f}s")
print(classification_report(y_test_shared, yp,
digits=4))
```

A.13 FIVE-FOLD CROSS-VALIDATION (All 9 Models, Identical Folds)

```
skf = StratifiedKFold(n_splits=5, shuffle=True,
random_state=SEED) all_models = ['SVM (RBF)', 'XGBoost',
'Random Forest',
        'Logistic Regression', 'QSVM (ZZ)',
        'QSVM (Pauli)', 'VQC', 'QNN', 'QKA']
cv_scores = {name: [] for name in
all_models} for fold, (train_idx,
val_idx) in enumerate(
    skf.split(X_train_q, y_train_shared)):
  Xtr, Xval = X_train_q[train_idx], X_train_q[val_idx]
  ytr, yval = y_train_shared[train_idx], y_train_shared[val_idx]
```

Classical

```
  for name, cls in [
    ('SVM (RBF)', SVC(kernel='rbf', gamma='scale',
                      random_state=SEED)),
```

```
('XGBoost', XGBClassifier(n_estimators=100,
            max_depth=4, random_state=SEED,
            use_label_encoder=False,
            eval_metric='logloss')),
('Random Forest',
          RandomForestClassifier(n_estimators=100,
          max_depth=8, random_state=SEED)),
('Logistic Regression',
            LogisticRegression(max_iter=1000,
            random_state=SEED))]:
cls.fit(Xtr, ytr)
cv_scores[name].append(
  f1_score(yval, cls.predict(Xval), zero_division=0))
```

QSVM-ZZ

```
fm =
        ZZFeatureMap(feature_dimension=NUM
        _QUBITS, reps=1, entanglement='linear')
qk =
FidelityQuantumKernel(feature_map
=fm) svc = SVC(kernel=qk.evaluate);
svc.fit(Xtr, ytr) cv_scores['QSVM
(ZZ)'].append(
  f1_score(yval, svc.predict(Xval), zero_division=0))
```

QSVM-Pauli

```
fm_p = PauliFeatureMap(feature_dimension=NUM_QUBITS, reps=1,
          paulis=['Z','ZZ'], entanglement='linear')
qk_p =
```

```
        FidelityQuantumKernel(feature_map=fm_p)
        svc_p = SVC(kernel=qk_p.evaluate);
        svc_p.fit(Xtr, ytr) cv_scores['QSVM
        (Pauli)'].append(
          f1_score(yval, svc_p.predict(Xval), zero_division=0))
```

VQC (50 iters for CV speed)

```
np.random.seed(SEED)
vr  = scipy_minimize(vqc_loss,
     np.random.uniform(-np.pi, np.pi,
     len(vqc_ans_params)), args=(Xtr, ytr),
     method='COBYLA',
     options={'maxiter': 50})
vp_cv = (vqc_forward(Xval, vr.x) >= 0.5).astype(int)
cv_scores['VQC'].append(
  f1_score(yval, vp_cv,
zero_division=0))
```

QNN (50 iters for CV speed)

```
np.random.seed(SEED)
qr  = scipy_minimize(qnn_loss,
     np.random.uniform(-np.pi, np.pi,
     len(qnn_ans_params)), args=(Xtr, ytr),
     method='COBYLA',
     options={'maxiter': 50})
qp_cv = (qnn_forward(Xval, qr.x) >= 0.5).astype(int)
cv_scores['QNN'].append(
  f1_score(yval, qp_cv, zero_division=0))
```

QKA

```
np.random.seed(
SEED) bfm  =
build_qka_circu
it(
    np.random.uniform(-np.pi, np.pi,
NUM_QUBITS   2)) qk_a  =
FidelityQuantumKernel(feature_map=bfm)
svc_a = SVC(kernel=qk_a.evaluate); svc_a.fit(Xtr, ytr)
cv_scores['QKA'].append(
        f1_score(yval, svc_a.predict(Xval),
      zero_division=0)) print(f"  Fold {fold+1}/5
      complete")
       CV Summary
     cv_df = pd.DataFrame({n: {'Mean': np.mean(s), 'Std':
              np.std(s)} for n, s in cv_scores.items()}).T
     print(cv_df.round(4))
```

QABD — QUANTUM ADVANTAGE BOUNDARY DETECTION

```
     sample_sizes = [50, 100, 150, 200]
     feature_dims = [2, 3, 4]
     qabd_q       =       np.zeros((len(sample_sizes),
     len(feature_dims)))              qabd_c              =
     np.zeros((len(sample_sizes),
     len(feature_dims))) print("Running QABD...")
     for i, ns in enumerate(sample_sizes):
       for j, nf in enumerate(feature_dims):
         pc  = PCA(n_components=nf,
         random_state=SEED) Xtr =
```

```
pc.fit_transform(X_train_balanced)
Xte = pc.transform(X_test_scaled)
np.random.seed(SEED)
sel = np.concatenate([
  np.random.choice(np.where(y_train_balanced==1)[0],
        ns//2, replace=False),
  np.random.choice(np.where(y_train_balanced==0)[0],
        ns//2,
replace=False)]) sc
= MinMaxScaler((-
1, 1))
Xtr_s  = sc.fit_transform(Xtr[sel])
ytr_s  = pd.Series(y_train_balanced).iloc[sel].values
Xte_s  = sc.transform(Xte)
np.random.seed(SEED)
st  = np.concatenate([fraud_test_idx,
    np.random.choice(genuine_test_idx,
        len(fraud_test_idx), replace=False)])
Xte_sub = Xte_s[st]; yte_sub = y_test_full.iloc[st].values
```

Quantum QSVM-ZZ

```
fm =
    ZZFeatureMap(feature_dimens
    ion=nf, reps=1,
    entanglement='linear')
qk  = FidelityQuantumKernel(feature_map=fm)
qs  = SVC(kernel=qk.evaluate); qs.fit(Xtr_s, ytr_s)
f1q = f1_score(yte_sub, qs.predict(Xte_sub),
```

```
    zero_division=0)    Classical RBF-SVM
    cs  = SVC(kernel='rbf',
    gamma='scale') cs.fit(Xtr_s,
    ytr_s)
    f1c = f1_score(yte_sub, cs.predict(Xte_sub),
    zero_division=0) qabd_q[i, j] = f1q
    qabd_c[i, j] = f1c
    m = "+" if f1q > f1c else
    "-" print(f"  [{m}]
    n={ns:3d} f={nf}: "
        f"Q={f1q:.3f} C={f1c:.3f} Δ={f1q-f1c:+.3f}")
qabd_adv = qabd_q - qabd_c
print(f"\nQuantum advantage in {(qabd_adv>0).sum()}/{qabd_adv.size} "
    f"configs | Max Δ =
{qabd_adv.max():+.4f}")     NOISE
ROBUSTNESS ANALYSIS
noise_levels = [0.00, 0.01, 0.02, 0.05,
0.10, 0.15]
        noise_f1   = {}
        for p in noise_levels:
          if p == 0:
            noise_f1[p] =
                    f1_score(y_test_shared,
                    qsvm_zz.predict(X_test_sha
                    red), zero_division=0)
          else:
            np.random.seed(SEED)
```

```
    yp    =
    qsvm_zz.predict(X_test_shared).cop
    y() flip_mask =
    np.random.random(len(yp)) < p    3
    yp[flip_mask] = 1 - yp[flip_mask]
    noise_f1[p]  = f1_score(y_test_shared, yp,
  zero_division=0) print(f"  p={p:.2f}: F1 =
  {noise_f1[p]:.4f}")
```

Noise robustness plot

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(noise_levels,
list(noise_f1.values()), 'o-',
    color='  6A1B9A', linewidth=2.5,
    markersize=10, label='QSVM-ZZ')
cf1 = results['SVM (RBF)']['F1']
ax.axhline(y=cf1, color='  1976D2', linestyle='--',
      linewidth=2, label=f'SVM-RBF baseline
      (immune): F1={cf1:.3f}')
ax.set_xlabel('Noise Probability'); ax.set_ylabel('F1 Score')
ax.set_title('Noise Robustness Analysis')
ax.legend(); ax.grid(True, alpha=0.3); ax.set_ylim(0, 1.05)
plt.tight_layout(); plt.show()
```

FINAL RESULTS TABLE

```
final_df =
pd.DataFrame(results).T.reindex(all_models)
final_df['Type']      = ['Classical']  4 +
['Quantum']  5
```

```
final_df['Train Time (s)'] = [round(timings[m], 1) for m in
all_models] final_df['CV F1 (mean±std)'] = [
f"{np.mean(cv_scores[m]):.3f}±{np.std(cv_scores[m]):.3f}"
  for m in all_models]
final_df = final_df[['Type', 'Recall',
            'Precision', 'F1', 'ROC-AUC',
            'PR-AUC', 'Accuracy',
            'CV F1 (mean±std)', 'Train Time
(s)']] print("="  100)
print("  FINAL RESULTS — Fair Same-Data
Comparison") print("="  100)
print(final_df.round(4).to_string())
```

Screenshots of Results & Visualizations

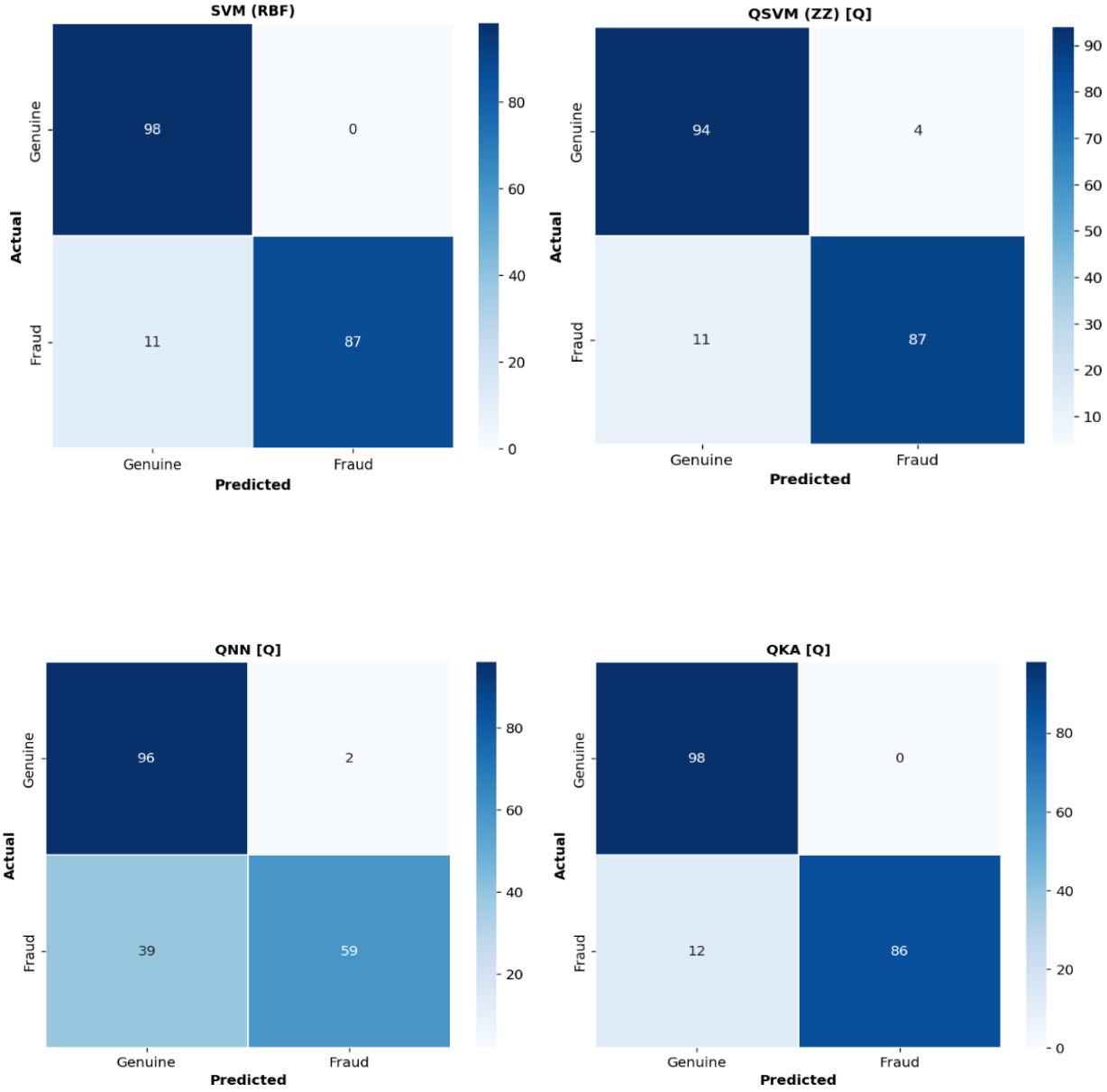

Fig B.1: Confusion Matrix Analysis (Selected Models)

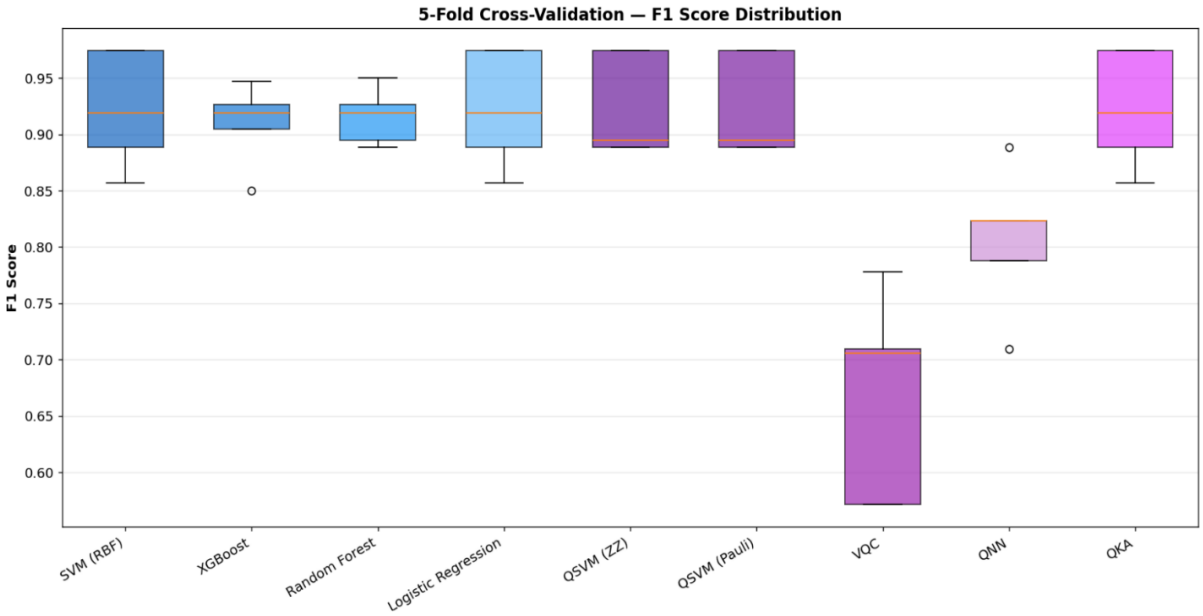

Fig B.2: Model Performance Comparison (5-Fold Cross-Validation)

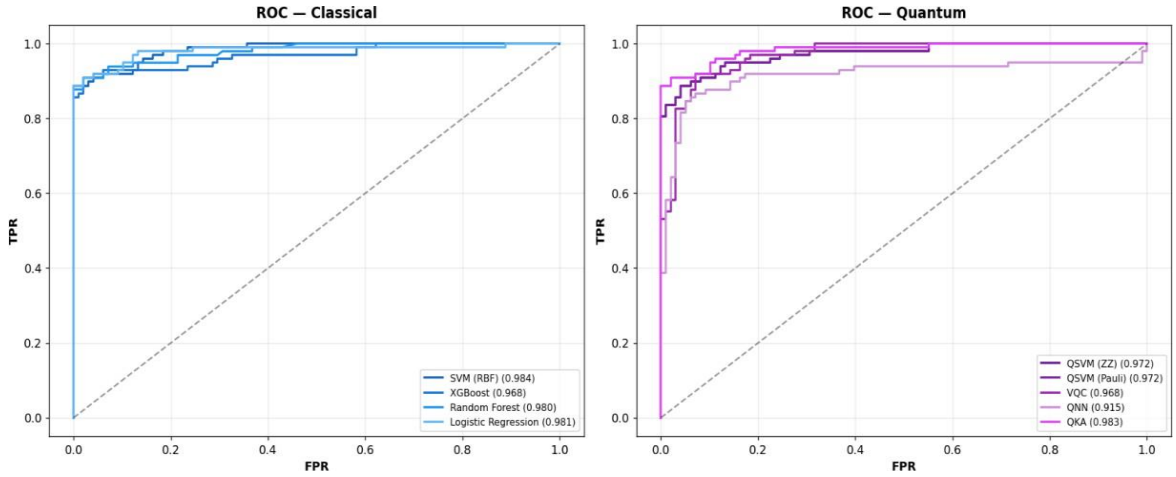

Fig B.3: ROC Curves for Classical and quantum models

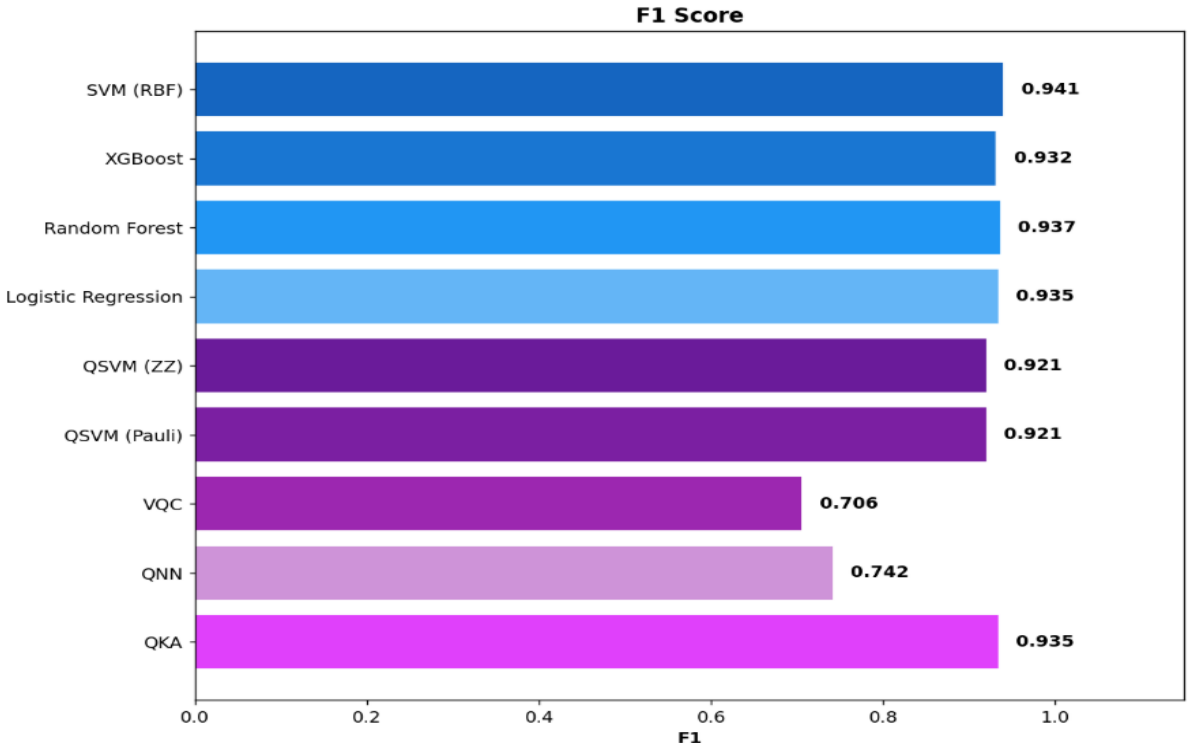

Fig B.4: F1-Score Comparison of Classical and Quantum Models

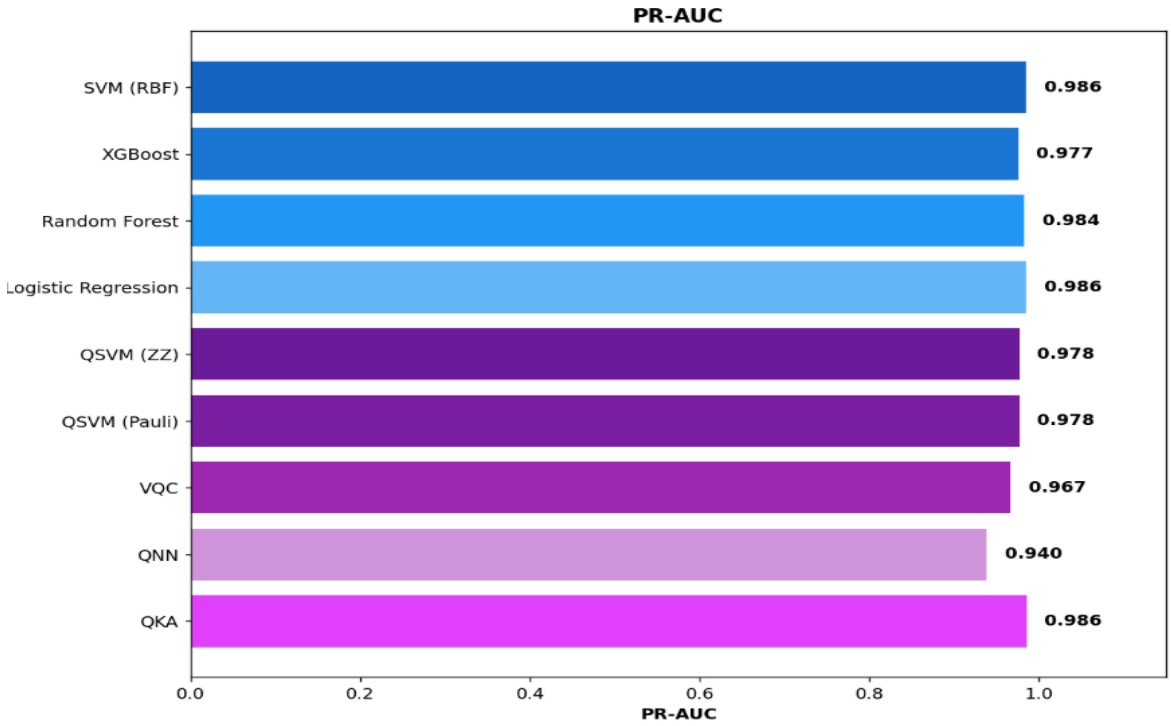

Fig B.5: PR-AUC Comparison of Classical and Quantum Models